70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · A2

GPU va sun'iy intellekt tezlatkichlaridan foydalangan holda parallel hisoblashni tashkil etish

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/A2
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/A2
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu virtual laboratoriyada siz matritsa ko'paytirish va konvolyutsiyani — neyron tarmoqning asosiy yadrolarini — bir necha darajada parallellashtirasiz va bir xil metodika (isinish, takror, median) bilan o'lchaysiz: sof Python → NumPy vektorlash → Numba @njit va prange (OpenMP muqobili) → kesh bloklash → PyTorch iplari → multiprocessing → Amdahl qonuni. Muhim: sandbox'da CUDA va GPU mavjud emas, shuning uchun real labdagi CUDA yadrolari va Tensor Core o'lchovlari o'rniga 2 yadroli CPU'da parallelizm o'rganiladi; har qadam oxirida «GPU bo'lsa shu kod qanday o'zgaradi» izohi berilgan. Massivlar kichik (n ≤ 1024), chunki Numba birinchi kompilyatsiyasi va 2 yadro vaqt chegarasini talab qiladi.

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/A2`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → A2 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Sof Python vs NumPy vektorlash
2. 2-qadam. Numba @njit va prange — OpenMP'ning Python muqobili
3. 3-qadam. Kesh bloklash (tiled) vs naive matritsa ko'paytirish
4. 4-qadam. Konvolyutsiyani to'rt usulda parallellashtirish
5. 5-qadam. PyTorch: GPU o'rniga ko'p yadro — 1 vs 2 ip, formatlar, yadro birlashtirish
6. 6-qadam. Ma'lumot parallelligi: multiprocessing bilan 2 jarayon
7. 7-qadam. Amdahl va Gustafson qonunlari: nazariya va 2 yadroda o'lchov
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Sof Python vs NumPy vektorlash

MAQSAD

Uch sikldagi sof Python matritsa ko'paytirish (n=64, 128) va NumPy BLAS chaqiruvi bir xil metodika bilan o'lchanadi, GFLOP/s va tezlanish hisoblanadi; so'ng 1M elementli GELU uchun list comprehension va vektorlangan NumPy solishtiriladi. Kutilgan natija: matmul'da 250–500×, GELU'da ~10× tezlanish; NumPy'ning oraliq massivlar tufayli xotira trafigi ko'rsatiladi. n va N ni o'zgartirib tezlanish qanday o'sishini kuzating.

> **Kutilgan natija:** Sof Python ~0,04 GFLOP/s, NumPy FP64 10–20 GFLOP/s, FP32 n=1024 da ~45 GFLOP/s; GELU 236 ms vs 21 ms.

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Sof Python vs NumPy vektorlash: matritsa ko'paytirish va elementlararo amal
# Bu yerda GPU yo'q (CUDA mavjud emas) — avval bitta yadroda interpretator va vektorlangan
# kutubxona o'rtasidagi farqni o'lchaymiz. O'lchov metodikasi: isinish + bir necha takror,
median.
import time
import random
import numpy as np

random.seed(0)
np.random.seed(0)


def median_vaqt(f, takror=3):
    """Funksiyani takror marta bajarib median vaqtni (s) qaytaradi (1 isinish)."""
    f()
    v = []
    for _ in range(takror):
        t0 = time.perf_counter()
        f()
        v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


# --- (a) Matritsa ko'paytirish ---
def matmul_python(A, B, n):
    C = [[0.0] * n for _ in range(n)]
    for i in range(n):
        Ai = A[i]
        Ci = C[i]
        for k in range(n):                # ikj tartib: B qatori bo'ylab ketma-ket murojaat
            a = Ai[k]
            Bk = B[k]
            for j in range(n):
                Ci[j] += a * Bk[j]
    return C


print("=== Sof Python vs NumPy ===")
print(f"{'variant':24s} {'n':>5s} {'vaqt, ms':>10s} {'GFLOP/s':>9s}")
natijalar = {}
for n in (64, 128):
    A = [[random.random() for _ in range(n)] for _ in range(n)]
```

```
    B = [[random.random() for _ in range(n)] for _ in range(n)]
    flop = 2 * n ** 3
    t_py = median_vaqt(lambda: matmul_python(A, B, n), takror=1 if n >= 128 else 3)
    An, Bn = np.array(A, dtype=np.float64), np.array(B, dtype=np.float64)
    t_np = median_vaqt(lambda: An @ Bn, takror=5)
    # to'g'rilikni tekshirish
    xato = np.abs(np.array(matmul_python(A, B, n)) - An @ Bn).max()
    print(f"{'sof Python (3 sikl)':24s} {n:5d} {t_py * 1e3:10.2f} {flop / t_py / 1e9:9.4f}")
    print(f"{'NumPy @ (BLAS, FP64)':24s} {n:5d} {t_np * 1e3:10.3f} {flop / t_np / 1e9:9.2f}")
    natijalar[n] = t_py / t_np
    print(f"  tezlanish {t_py / t_np:8.0f}x, maks xato {xato:.1e}")

for n in (512, 1024):
    An = np.random.rand(n, n).astype(np.float32)
    Bn = np.random.rand(n, n).astype(np.float32)
    t_np = median_vaqt(lambda: An @ Bn, takror=5)
    print(f"{'NumPy @ (BLAS, FP32)':24s} {n:5d} {t_np * 1e3:10.2f} {2 * n ** 3 / t_np /
1e9:9.2f}")

# --- (b) Elementlararo amal: GELU faollashtirish 1M element ---
N = 1_000_000
x_list = [random.uniform(-3, 3) for _ in range(N)]
x_np = np.array(x_list, dtype=np.float32)
import math

def gelu_python(xs):
    return [0.5 * x * (1 + math.tanh(0.79788456 * (x + 0.044715 * x ** 3))) for x in xs]

def gelu_numpy(x):
    return 0.5 * x * (1 + np.tanh(0.79788456 * (x + 0.044715 * x ** 3)))

t_py = median_vaqt(lambda: gelu_python(x_list), takror=1)
t_np = median_vaqt(lambda: gelu_numpy(x_np), takror=5)
print(f"\n--- GELU, {N / 1e6:.0f}M element ---")
print(f"sof Python (list comprehension): {t_py * 1e3:8.1f} ms  ({N / t_py / 1e6:6.1f} M
element/s)")
print(f"NumPy (vektorlangan):            {t_np * 1e3:8.1f} ms  ({N / t_np / 1e6:6.1f} M
element/s)  tezlanish {t_py / t_np:.0f}x")
# NumPy'da har amal alohida o'tish: ~8 ta oraliq massiv (har biri 4 MB) yoziladi va o'qiladi
trafik = 8 * 2 * N * 4
print(f"NumPy samarali xotira trafigi: ~{trafik / 1e6:.0f} MB -> {trafik / t_np / 1e9:.1f} GB/s
(yadro birlashtirilmagan)")
print("\nXulosa: interpretator 1 element uchun ~100 ns sarflaydi; BLAS/vektor buyruqlar 100-
1000x tez.")
print("GPU bo'lsa: shu NumPy kodi CuPy'da o'zgarishsiz ishlaydi, lekin 1M element uchun yadro
ishga tushirish xarajati (~10 mks) sezilarli.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Sof Python vs NumPy ===`, `GELU`

TAJRIBA QILIB KO‘RING

- matmul_python'ni ijk tartibiga o'zgartiring — sof Python'da ham kesh ta'siri sezilarmi?
- GELU'ni np.float64 bilan qayta o'lchang: xotira trafigi 2 baravar oshganda vaqt qanday o'zgaradi?
- NumPy matmul uchun n=2048 qo'shing va GFLOP/s barqaror qoladimi, tekshiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. Numba @njit va prange — OpenMP'ning Python muqobili

MAQSAD

@njit matmul (ikj) kompilyatsiya qilinadi, parallel=True + prange varianti 1 va 2 ipda (numba.set_num_threads) o'lchanadi, NumPy BLAS bilan solishtiriladi; alohida reduktsiya (sum x²) misoli 2 ipda xotira o'tkazuvchanligi cheklovini ko'rsatadi. Kutilgan natija: JIT kompilyatsiya ~0,3–1 s, prange 2 ipda ~1,6–1,9× tezlanish, BLAS yana 1,5× tezroq. Bu real labdagi `#pragma omp parallel for` va `reduction(+:s)` ning to'g'ridan-to'g'ri muqobili.

> **Kutilgan natija:** Kompilyatsiya vaqtlari, threading layer nomi, 4 qatorli jadval (seq ~16 GFLOP/s, prange 2 ip ~27, BLAS ~39), reduktsiya 11 → 22 GB/s.

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. Numba @njit va prange — OpenMP'ning Python muqobili
# @njit kodni mashina kodiga kompilyatsiya qiladi (JIT), parallel=True + prange esa siklni
# iplar orasida taqsimlaydi (OpenMP `#pragma omp parallel for` kabi). Sandbox'da 2 yadro bor.
import time
import numpy as np
import numba
from numba import njit, prange

numba.set_num_threads(2)


@njit(cache=False)
def matmul_seq(A, B, C):
    n, m, p = A.shape[0], A.shape[1], B.shape[1]
    for i in range(n):
        for k in range(m):                # ikj: B qatori bo'ylab ketma-ket murojaat (kesh
uchun qulay)
            a = A[i, k]
            for j in range(p):
                C[i, j] += a * B[k, j]


@njit(parallel=True, cache=False)
def matmul_par(A, B, C):
    n, m, p = A.shape[0], A.shape[1], B.shape[1]
    for i in prange(n):                   # tashqi sikl iplar orasida taqsimlanadi
(schedule(static) kabi)
        for k in range(m):
            a = A[i, k]
            for j in range(p):
                C[i, j] += a * B[k, j]


@njit(parallel=True, cache=False)
def yigindi_par(x):
    s = 0.0
    for i in prange(x.shape[0]):          # reduction(+:s) ga ekvivalent — numba avtomatik
aniqlaydi
        s += x[i] * x[i]
    return s
```

```
def olchov(f, *args, takror=5):
    v = []
    for _ in range(takror):
        args[-1][:] = 0.0
        t0 = time.perf_counter()
        f(*args)
        v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


n = 256
A = np.random.rand(n, n).astype(np.float32)
B = np.random.rand(n, n).astype(np.float32)
C = np.zeros((n, n), dtype=np.float32)
flop = 2 * n ** 3

print("=== Numba njit va prange ===")
print(f"Numba {numba.__version__}, iplar: {numba.get_num_threads()}, threading layer
tanlanadi...")

# Birinchi chaqiruv = kompilyatsiya vaqti (OpenMP'da bu kompilyator vaqti, bu yerda runtime'da)
t0 = time.perf_counter(); matmul_seq(A, B, C); t_komp_seq = time.perf_counter() - t0
C[:] = 0
t0 = time.perf_counter(); matmul_par(A, B, C); t_komp_par = time.perf_counter() - t0
print(f"JIT kompilyatsiya: seq {t_komp_seq:.2f} s, parallel {t_komp_par:.2f} s (bir marta,
keyin keshdan)")
print(f"Threading layer: {numba.threading_layer()}")

# --- Python etalon (kichik n) bilan to'g'rilik ---
etalon = A @ B
C[:] = 0; matmul_par(A, B, C)
print(f"Maks xato (par vs NumPy): {np.abs(C - etalon).max():.2e}")

print(f"\n{'variant':22s} {'iplar':>5s} {'vaqt, ms':>9s} {'GFLOP/s':>8s} {'tezlanish':>9s}")
t_seq = olchov(matmul_seq, A, B, C)
print(f"{'njit seq (ikj)':22s} {1:5d} {t_seq * 1e3:9.2f} {flop / t_seq / 1e9:8.2f} {1.0:9.2f}")
for iplar in (1, 2):
    numba.set_num_threads(iplar)
    t = olchov(matmul_par, A, B, C)
    print(f"{'njit parallel prange':22s} {iplar:5d} {t * 1e3:9.2f} {flop / t / 1e9:8.2f} {t_seq
/ t:9.2f}")
numba.set_num_threads(2)
t_np = olchov(lambda a, b, c: np.matmul(a, b, out=c), A, B, C)
print(f"{'NumPy BLAS (2 ip)':22s} {2:5d} {t_np * 1e3:9.2f} {flop / t_np / 1e9:8.2f} {t_seq /
t_np:9.2f}")

# --- Reduktsiya (yig'indi) 1 vs 2 ip ---
x = np.random.rand(4_000_000).astype(np.float64)
yigindi_par(x)                                  # kompilyatsiya
print("\n--- Reduktsiya: sum(x*x), 4M element ---")
for iplar in (1, 2):
    numba.set_num_threads(iplar)
    v = []
    for _ in range(7):
        t0 = time.perf_counter(); s = yigindi_par(x); v.append(time.perf_counter() - t0)
    t = sorted(v)[3]
    print(f"iplar={iplar}: {t * 1e3:7.2f} ms, {2 * x.nbytes / t / 1e9:5.1f} GB/s samarali
(o'qish), natija {s:.1f}")
print("Reduktsiya xotira bilan cheklangan: 2 ip hisobni emas, DRAM o'tkazuvchanligini
bo'lishadi.")
print("\nGPU bo'lsa: @njit o'rniga @cuda.jit, prange o'rniga cuda.grid(2) indekslari, massivlar
cuda.to_device bilan ko'chiriladi,")
print("vaqt o'lchashdan oldin cuda.synchronize() shart.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Numba njit va prange ===`, `Reduktsiya`

TAJRIBA QILIB KO‘RING

- prange'ni ichki (j) siklga ko'chiring — tezlanish nima uchun yo'qoladi (ip yaratish xarajati)?
- n=512 va n=1024 uchun o'lchab, 2 ip samaradorligi (tezlanish/2) o'lcham bilan qanday o'zgarishini jadvalga yozing.
- fastmath=True bayrog'ini qo'shing va GFLOP/s hamda maks xato qanday o'zgarishini solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Kesh bloklash (tiled) vs naive matritsa ko'paytirish

MAQSAD

Uch algoritm bir ipda solishtiriladi: naive ijk (B ustun bo'ylab — har qadam yangi kesh satri), ikj (qator bo'ylab, vektorlashadi) va bloklangan (tiled) variant; so'ng blok o'lchami 8…256 ta'siri o'lchanadi. Kutilgan natija: ijk → ikj ~10× tezlanish; kichik bloklar sikl xarajati tufayli sekin, 128–256 da ikj darajasiga yetadi. Kodda Numba'ning muhim nozik jihati ko'rsatilgan: range(jj, jj+bs) indekslari manfiy-tekshiruv tufayli vektorlashmaydi, slice bilan hal qilinadi.

> **Kutilgan natija:** n=256/512/1024 jadvali: naive ~1,7 GFLOP/s, ikj ~15–17, tiled ~12; blok o'lchami jadvali 5 → 15 GFLOP/s.

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Kesh bloklash (cache tiling): naive (ijk) vs ikj vs bloklangan (tiled) matritsa
ko'paytirish
# GPU'dagi umumiy xotirali (shared memory) yadroning CPU muqobili — blok L1/L2 keshga sig'adi
va har
# yuklangan element ko'p marta qayta ishlatiladi. Numba bilan 1 ipda o'lchaymiz (algoritm
farqini ko'rish uchun).
import time
import numpy as np
from numba import njit

BS = 64   # blok o'lchami (o'zgartirib ko'ring: 16, 32, 128)


@njit
def matmul_ijk(A, B, C):
    n = A.shape[0]
    for i in range(n):
        for j in range(n):
            s = 0.0
            for k in range(n):            # B[k, j] — ustun bo'ylab, har qadam yangi kesh
satri
                s += A[i, k] * B[k, j]
            C[i, j] = s


@njit
def matmul_ikj(A, B, C):
    n = A.shape[0]
    for i in range(n):
        for k in range(n):
            a = A[i, k]
            for j in range(n):            # B[k, j] — qator bo'ylab ketma-ket, vektorlashadi
                C[i, j] += a * B[k, j]


@njit
def matmul_tiled(A, B, C, bs):
    """Bloklangan ikj. Muhim nozik jihat: numba `range(jj, jj+bs)` indekslarini manfiy bo'lishi
mumkin deb hisoblab
    har murojaatga tekshiruv qo'shadi va ichki sikl vektorlashmaydi (2 GFLOP/s). Qator
bo'lagini (slice) olib,
    0 dan boshlanadigan indeks bilan yursak — vektorlashadi. n bs ga karrali deb olinadi."""
```

```
    n = A.shape[0]
    for ii in range(0, n, bs):
        for kk in range(0, n, bs):
            for jj in range(0, n, bs):
                for i in range(ii, ii + bs):
                    Ci = C[i, jj:jj + bs]
                    for k in range(kk, kk + bs):
                        a = A[i, k]
                        Bk = B[k, jj:jj + bs]
                        for j in range(bs):
                            Ci[j] += a * Bk[j]


def olchov(f, A, B, C, *extra, takror=3):
    v = []
    for _ in range(takror):
        C[:] = 0
        t0 = time.perf_counter()
        f(A, B, C, *extra)
        v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


print("=== Kesh bloklash ===")
# kompilyatsiya (kichik massivda)
a0 = np.ones((8, 8), np.float32); c0 = np.zeros((8, 8), np.float32)
matmul_ijk(a0, a0, c0); matmul_ikj(a0, a0, c0); matmul_tiled(a0, a0, c0, 4)

print(f"{'variant':16s} {'n':>5s} {'xotira, MB':>10s} {'vaqt, ms':>9s} {'GFLOP/s':>8s}
{'xato':>8s}")
for n in (256, 512, 1024):
    A = np.random.rand(n, n).astype(np.float32)
    B = np.random.rand(n, n).astype(np.float32)
    C = np.zeros((n, n), np.float32)
    etalon = A @ B
    flop = 2 * n ** 3
    mb = 3 * n * n * 4 / 1e6
    for nom, f, extra in (("naive ijk", matmul_ijk, ()), ("ikj", matmul_ikj, ()), (f"tiled bs=
{BS}", matmul_tiled, (BS,))):
        if nom == "naive ijk" and n > 512:
            continue                        # sodda variant katta n da juda sekin (bir necha
soniya)
        t = olchov(f, A, B, C, *extra, takror=1 if n >= 512 else 3)
        print(f"{nom:16s} {n:5d} {mb:10.1f} {t * 1e3:9.1f} {flop / t / 1e9:8.2f} {np.abs(C -
etalon).max():8.1e}")

# Blok o'lchami ta'siri (n=1024)
n = 1024
A = np.random.rand(n, n).astype(np.float32); B = np.random.rand(n, n).astype(np.float32); C =
np.zeros((n, n), np.float32)
print(f"\n--- Blok o'lchami ta'siri, n={n} ---")
print(f"{'bs':>4s} {'blok hajmi (3 blok), KB':>24s} {'vaqt, ms':>9s} {'GFLOP/s':>8s}")
for bs in (8, 16, 32, 64, 128, 256):
    t = olchov(matmul_tiled, A, B, C, bs, takror=1)
    print(f"{bs:4d} {3 * bs * bs * 4 / 1024:24.1f} {t * 1e3:9.1f} {2 * n ** 3 / t / 1e9:8.2f}")
print("\nArifmetik intensivlik: ijk ~ 0.25 FLOP/bayt (har 2 FLOP uchun 8 bayt), tiled ~ bs/6
FLOP/bayt.")
print("Eslatma: bu serverda L3 kesh katta, shuning uchun n=1024 da ikj ham keshdan ishlaydi va
tiled undan oshmasligi mumkin;")
print("bloklash foydasi matritsa keshdan katta bo'lganda (n>=4096, 64+ MB) va GPU umumiy
xotirasida yaqqol ko'rinadi.")
print("GPU bo'lsa: blok = cuda.shared.array((TPB,TPB)), yuklash -> cuda.syncthreads() -> hisob;
TPB=16 da har element 16 marta qayta ishlatiladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Kesh bloklash ===`, `Blok o'lchami ta'siri`

TAJRIBA QILIB KO‘RING

- matmul_tiled'da slice o'rniga to'g'ridan-to'g'ri C[i, j] indekslashga qaytib, GFLOP/s 2 ga tushishini o'zingiz tekshiring.
- n=2048 (48 MB, keshdan katta) uchun faqat ikj va tiled bs=128 ni o'lchang — endi bloklash foyda beradimi?
- Blokni ikki darajali qiling (tashqi 256, ichki 32) va natijani yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Konvolyutsiyani to'rt usulda parallellashtirish

MAQSAD

3×3 konvolyutsiya (16→32 kanal, 96×96) Numba to'g'ridan-to'g'ri sikl (1 ip), prange (1 va 2 ip), NumPy im2col+GEMM va PyTorch conv2d (oneDNN, 1 va 2 ip, batch 8) bilan bajariladi; barcha natijalar etalon bilan solishtiriladi. Kutilgan natija: prange 2 ipda ~2×, im2col ~16×, torch 2 ip ~20× tezlanish; xato < 1e-5. Bu cuDNN'ning implicit GEMM g'oyasini CPU'da ko'rsatadi.

> **Kutilgan natija:** 81,4 MFLOP; jadval: Numba 1 ip ~2 GFLOP/s, prange 2 ip ~4, im2col ~33, torch 2 ip ~46 GFLOP/s; maks xato ~5e-6.

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Konvolyutsiya (3x3) ni to'rt usulda: Numba to'g'ridan-to'g'ri (1 ip), Numba prange
(2 ip),
# NumPy im2col + GEMM, PyTorch conv2d (oneDNN, 2 ip). Hammasi bir xil natija berishi
tekshiriladi.
import time
import numpy as np
import numba
from numba import njit, prange
import torch
import torch.nn.functional as F

torch.set_num_threads(2)
torch.backends.nnpack.set_flags(False)
numba.set_num_threads(2)
np.random.seed(0)

CIN, COUT, H, W, K = 16, 32, 96, 96, 3
HO, WO = H - K + 1, W - K + 1
FLOP = 2 * K * K * CIN * COUT * HO * WO


@njit
def conv_seq(x, w, y):
    cin, h, wd = x.shape
    cout, ho, wo = y.shape
    for co in range(cout):
        for i in range(ho):
            for j in range(wo):
                s = 0.0
                for ci in range(cin):
                    for di in range(3):
                        for dj in range(3):
                            s += x[ci, i + di, j + dj] * w[co, ci, di, dj]
                y[co, i, j] = s


@njit(parallel=True)
def conv_par(x, w, y):
    cin, h, wd = x.shape
    cout, ho, wo = y.shape
    for co in prange(cout):                # chiqish kanallari iplar orasida taqsimlanadi
        for i in range(ho):
            for j in range(wo):
```

```
                s = 0.0
                for ci in range(cin):
                    for di in range(3):
                        for dj in range(3):
                            s += x[ci, i + di, j + dj] * w[co, ci, di, dj]
                y[co, i, j] = s


def conv_im2col(x, w):
    """Konvolyutsiyani matritsa ko'paytirishga keltirish (cuDNN'ning implicit GEMM g'oyasi)."""
    cols = np.empty((CIN * K * K, HO * WO), dtype=np.float32)
    r = 0
    for ci in range(CIN):
        for di in range(K):
            for dj in range(K):
                cols[r] = x[ci, di:di + HO, dj:dj + WO].ravel()
                r += 1
    return (w.reshape(COUT, -1) @ cols).reshape(COUT, HO, WO)


def olchov(f, takror=5):
    f()
    v = []
    for _ in range(takror):
        t0 = time.perf_counter(); f(); v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


x = np.random.rand(CIN, H, W).astype(np.float32)
w = (np.random.rand(COUT, CIN, K, K).astype(np.float32) - 0.5)
y1 = np.zeros((COUT, HO, WO), np.float32)
y2 = np.zeros((COUT, HO, WO), np.float32)
xt, wt = torch.from_numpy(x)[None], torch.from_numpy(w)

print("=== Konvolyutsiya ===")
print(f"Kirish {CIN}x{H}x{W}, yadro {COUT}x{CIN}x{K}x{K}, chiqish {COUT}x{HO}x{WO}: {FLOP /
1e6:.1f} MFLOP")
t0 = time.perf_counter(); conv_seq(x, w, y1); conv_par(x, w, y2); print(f"Numba kompilyatsiya:
{time.perf_counter() - t0:.2f} s")

etalon = F.conv2d(xt, wt)[0].numpy()
natija = []
natija.append(("Numba to'g'ridan-to'g'ri, 1 ip", olchov(lambda: conv_seq(x, w, y1)), y1))
numba.set_num_threads(1)
natija.append(("Numba prange, 1 ip", olchov(lambda: conv_par(x, w, y2)), y2))
numba.set_num_threads(2)
natija.append(("Numba prange, 2 ip", olchov(lambda: conv_par(x, w, y2)), y2))
natija.append(("NumPy im2col + GEMM", olchov(lambda: conv_im2col(x, w)), conv_im2col(x, w)))
with torch.no_grad():
    torch.set_num_threads(1)
    natija.append(("torch conv2d, 1 ip", olchov(lambda: F.conv2d(xt, wt)), etalon))
    torch.set_num_threads(2)
    natija.append(("torch conv2d, 2 ip", olchov(lambda: F.conv2d(xt, wt)), etalon))
    xb = xt.repeat(8, 1, 1, 1)
    t_b = olchov(lambda: F.conv2d(xb, wt))
    natija.append(("torch conv2d, 2 ip, batch 8", t_b / 8, etalon))

print(f"\n{'usul':30s} {'ms':>8s} {'GFLOP/s':>8s} {'tezlanish':>9s} {'maks xato':>10s}")
t_bazaviy = natija[0][1]
for nom, t, yy in natija:
    print(f"{nom:30s} {t * 1e3:8.2f} {FLOP / t / 1e9:8.2f} {t_bazaviy / t:9.1f} {np.abs(yy -
etalon).max():10.1e}")
print("\nXulosa: to'g'ridan-to'g'ri sikl -> prange (2x) -> GEMM'ga keltirish (BLAS) -> oneDNN
```

```
(vektorlash + bloklash).")
print("GPU bo'lsa: F.conv2d cuDNN'ga o'tadi (implicit GEMM/Winograd), FP16'da Tensor Core;
o'lchash uchun torch.cuda.synchronize() kerak.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Konvolyutsiya ===`, `torch conv2d`

TAJRIBA QILIB KO‘RING

- prange'ni chiqish kanallari (co) o'rniga qatorlar (i) bo'yicha qo'ying — yuk muvozanati o'zgaradimi?
- Yadroni 5×5 qilib FLOP va vaqtni qayta o'lchang; im2col matritsasi hajmi necha baravar oshadi?
- CIN=64, COUT=64, 224×224 (mustaqil ish topshirig'i) uchun faqat torch variantini o'lchang va GFLOP/s ni solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. PyTorch: GPU o'rniga ko'p yadro — 1 vs 2 ip, formatlar, yadro birlashtirish

MAQSAD

torch.cuda.is_available() False ekani ko'rsatiladi va GPU'dagi o'zgarishlar izohlanadi. CPU'da: (a) matmul n=128/512/1024 uchun torch.set_num_threads(1) vs (2) tezlanishi; (b) FP32 vs BF16 vs INT8 (torch._int_mm) — CPU'da maxsus buyruqlar (AVX512-BF16/VNNI) bo'lmasa past formatlar sekinroq, bu apparat–format mosligi (Co-design) darsidir; (c) GELU eager (7 alohida amal) vs birlashtirilgan F.gelu — xotira trafigi 7× kam. Kutilgan natija: 2 ip katta n da ~2×, BF16 CPU'da 20× sekin, F.gelu ~3× tez.

> **Kutilgan natija:** Iplar jadvali (1024 da 1,98× tezlanish), formatlar jadvali (FP32 ~50 GFLOP/s, BF16 ~2, INT8 ~4), GELU: eager ~130–220 ms, F.gelu ~45–80 ms.

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. PyTorch: GPU o'rniga ko'p yadro — torch.set_num_threads(1) vs 2, formatlar va yadro
birlashtirish
# Sandbox'da CUDA mavjud emas (torch.cuda.is_available() == False). Real labdagi Tensor Core
o'lchovlari
# o'rniga CPU'da: (a) iplar soni ta'siri, (b) FP32 vs BF16 vs INT8 (CPU'da!), (c) GELU eager vs
birlashtirilgan.
import time
import torch
import torch.nn.functional as F

torch.manual_seed(0)
torch.backends.nnpack.set_flags(False)
print("=== torch iplar: 1 vs 2 ===")
print(f"torch {torch.__version__}, CUDA mavjud: {torch.cuda.is_available()}, CPU iplar
(standart): {torch.get_num_threads()}")
print("GPU bo'lsa: dev='cuda', tensorlar .to(dev), vaqt oldidan torch.cuda.synchronize();
FP16/BF16 Tensor Core'ga o'tadi.\n")


def olchov(f, takror=7):
    for _ in range(2):
        f()
    v = []
    for _ in range(takror):
        t0 = time.perf_counter(); f(); v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]


# --- (a) Matritsa ko'paytirish: iplar soni va o'lcham ---
print(f"{'n':>5s} {'iplar':>5s} {'ms':>9s} {'GFLOP/s':>8s} {'tezlanish':>9s}")
for n in (128, 512, 1024):
    A = torch.randn(n, n); B = torch.randn(n, n)
    t1 = None
    for iplar in (1, 2):
        torch.set_num_threads(iplar)
        t = olchov(lambda: A @ B)
        if t1 is None:
            t1 = t
```

```
        print(f"{n:5d} {iplar:5d} {t * 1e3:9.3f} {2 * n ** 3 / t / 1e9:8.1f} {t1 / t:9.2f}")
print("Kichik n da 2 ip foyda bermaydi (ip sinxronizatsiyasi xarajati), katta n da ~2x ga
yaqinlashadi.")

# --- (b) Formatlar: FP32 vs BF16 vs INT8 — CPU'da tezlanish bormi? ---
torch.set_num_threads(2)
n = 512
A = torch.randn(n, n); B = torch.randn(n, n)
etalon = A @ B
print(f"\n--- Formatlar, n={n}, 2 ip (CPU) ---")
print(f"{'format':8s} {'ms':>9s} {'GFLOP/s':>8s} {'nisbiy xato':>12s}")
t32 = olchov(lambda: A @ B)
print(f"{'FP32':8s} {t32 * 1e3:9.3f} {2 * n ** 3 / t32 / 1e9:8.1f} {0.0:12.1e}")
Ab, Bb = A.bfloat16(), B.bfloat16()
tb = olchov(lambda: Ab @ Bb)
xb = ((Ab @ Bb).float() - etalon).norm() / etalon.norm()
print(f"{'BF16':8s} {tb * 1e3:9.3f} {2 * n ** 3 / tb / 1e9:8.1f} {xb.item():12.1e}")
# INT8: dinamik kvantlash (A3 mavzusi): masshtab * int8 matritsa
sa, sb = A.abs().max() / 127, B.abs().max() / 127
A8 = (A / sa).round().clamp(-128, 127).to(torch.int8)
B8 = (B / sb).round().clamp(-128, 127).to(torch.int8)
try:
    t8 = olchov(lambda: torch._int_mm(A8, B8))
    x8 = ((torch._int_mm(A8, B8).float() * sa * sb) - etalon).norm() / etalon.norm()
    print(f"{'INT8':8s} {t8 * 1e3:9.3f} {2 * n ** 3 / t8 / 1e9:8.1f} {x8.item():12.1e}
(torch._int_mm)")
except Exception as e:
    print(f"{'INT8':8s} CPU'da _int_mm qo'llab-quvvatlanmaydi: {type(e).__name__}")
print("CPU'da BF16/INT8 tezlanish bermasligi mumkin (AVX512-BF16/VNNI yo'q) — GPU Tensor
Core'da 2-8x. Bu apparat-format moslik (Co-design) misoli.")

# --- (c) Elementlararo amal: GELU eager (6-7 alohida yadro) vs birlashtirilgan F.gelu ---
N = 8 * 1024 * 1024                                  # 8M element = 32 MB FP32
x = torch.randn(N)
def gelu_eager(x):
    return 0.5 * x * (1 + torch.tanh(0.79788456 * (x + 0.044715 * x ** 3)))
print(f"\n--- GELU, {N // 2 ** 20}M element ---")
print(f"{'variant':22s} {'iplar':>5s} {'ms':>8s} {'samarali GB/s':>14s}")
for iplar in (1, 2):
    torch.set_num_threads(iplar)
    for nom, f, otish in (("eager (7 yadro)", lambda: gelu_eager(x), 7 * 2), ("F.gelu (1
yadro)", lambda: F.gelu(x, approximate="tanh"), 2)):
        t = olchov(f, takror=5)
        print(f"{nom:22s} {iplar:5d} {t * 1e3:8.2f} {otish * N * 4 / t / 1e9:14.1f}")
print("Eager rejimda har amal 32 MB o'qib-yozadi; birlashtirilgan yadro bitta o'tishda — xotira
trafigi 7x kam.")
print("GPU bo'lsa: torch.compile Triton bilan shunday birlashtirishni avtomatik qiladi
(sandbox'da C kompilyatori yo'q).")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== torch iplar: 1 vs 2 ===`, `GELU`

TAJRIBA QILIB KO‘RING

- n=4097 kabi 8 ga karrali bo'lmagan o'lchamni qo'shing — CPU'da ham unumdorlik tushadimi (GPU'dagi Tensor Core sharti bilan solishtiring)?
- GELU uchun N ni 1M ga tushiring: eager va fused farqi saqlanadimi (kesh ichida ishlaganda)?
- torch.float16 matmul'ni CPU'da sinab ko'ring va BF16 bilan vaqt/xato bo'yicha solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Ma'lumot parallelligi: multiprocessing bilan 2 jarayon

MAQSAD

600 ta sintetik kadr uchun Python+NumPy oldindan ishlov (Sobel, filtr, gistogramma) ketma-ket va multiprocessing.Pool (fork, 1 va 2 jarayon, chunksize 1/25/150) bilan bajariladi; tezlanish, samaradorlik va Pool yaratish xarajati o'lchanadi, natija ketma-ket bilan taqqoslanib tekshiriladi. Kutilgan natija: 2 jarayonda ~1,9–2,4× (chunksize kattaroq bo'lsa yaxshiroq), Pool ochish ~10–50 ms. Bu DDP'dagi «har qurilma o'z ulushini ishlaydi» g'oyasining CPU muqobili.

> **Kutilgan natija:** Ketma-ket ~0,6 s; jadval 6 qator (jarayonlar × chunksize), 2 jarayonda ~2× tezlanish; Pool xarajati xabari.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Ma'lumot parallelligi: multiprocessing.Pool bilan kadrlarni 2 jarayonga bo'lish
# Python GIL tufayli iplar CPU-og'ir Python kodini tezlashtirmaydi; jarayonlar (fork) esa
# har biri o'z interpretatoriga ega. Bu ko'p-GPU DDP'dagi "har qurilma o'z ulushini ishlaydi"
g'oyasining CPU muqobili.
import os
import time
import multiprocessing as mp
import numpy as np

KADRLAR = 600                  # 'video kadrlar' soni
H = W = 96


def kadr_ishlov(kadr):
    """Bitta kadr uchun oldindan ishlov: normalizatsiya, Sobel gradient, 3x3 o'rtacha filtr,
gistogramma.
    Kichik NumPy massivlar ustida ko'p amal — BLAS iplari ishlamaydi, bu Python+NumPy
yukidir."""
    x = kadr.astype(np.float32) / 255.0
    gx = np.zeros_like(x); gy = np.zeros_like(x)
    for _ in range(6):                                  # ishni og'irlashtirish (real quvurda
bir nechta bosqich)
        gx[:, 1:-1] = x[:, 2:] - x[:, :-2]
        gy[1:-1, :] = x[2:, :] - x[:-2, :]
        mag = np.sqrt(gx * gx + gy * gy)
        x = (x[:-2, :-2] + x[1:-1, :-2] + x[2:, :-2] + x[:-2, 1:-1] + x[1:-1, 1:-1] + x[2:,
1:-1]
             + x[:-2, 2:] + x[1:-1, 2:] + x[2:, 2:]) / 9.0
        x = np.pad(x, 1)
    hist, _ = np.histogram(mag, bins=16, range=(0, 1))
    return float(mag.mean()), hist.argmax()


def ketma_ket(kadrlar):
    return [kadr_ishlov(k) for k in kadrlar]


if __name__ == "__main__":
    rng = np.random.default_rng(0)
    kadrlar = [rng.integers(0, 256, (H, W), dtype=np.uint8) for _ in range(KADRLAR)]

    print("=== Ma'lumot parallelligi (multiprocessing) ===")
```

```
    print(f"CPU yadrolar (ko'rinadigan): {os.cpu_count()}, ruxsat etilgan:
{len(os.sched_getaffinity(0))}, kadrlar: {KADRLAR}")

    t0 = time.perf_counter(); etalon = ketma_ket(kadrlar); t_seq = time.perf_counter() - t0
    print(f"\nKetma-ket (1 jarayon): {t_seq * 1e3:7.0f} ms  ({KADRLAR / t_seq:.0f} kadr/s)")

    ctx = mp.get_context("fork")
    print(f"\n{'jarayonlar':>10s} {'chunksize':>9s} {'ms':>7s} {'kadr/s':>7s} {'tezlanish':>9s}
{'samaradorlik':>12s}")
    for n_proc in (1, 2):
        for chunksize in (1, 25, 150):
            with ctx.Pool(n_proc) as pool:
                pool.map(kadr_ishlov, kadrlar[:20])          # isinish
                t0 = time.perf_counter()
                natija = pool.map(kadr_ishlov, kadrlar, chunksize=chunksize)
                t = time.perf_counter() - t0
            assert natija == etalon, "natija mos emas!"
            print(f"{n_proc:10d} {chunksize:9d} {t * 1e3:7.0f} {KADRLAR / t:7.0f} {t_seq /
t:9.2f} {t_seq / t / n_proc * 100:11.0f} %")

    # Jarayon yaratish xarajati (DDP'da bu 'process group init' ga o'xshaydi)
    t0 = time.perf_counter()
    with ctx.Pool(2) as pool:
        pool.map(kadr_ishlov, kadrlar[:2])
    print(f"\nPool(2) yaratish + yopish xarajati: {(time.perf_counter() - t0) * 1e3:.0f} ms —
qisqa vazifalar uchun parallelizm o'zini oqlamaydi.")
    print("chunksize=1 da har kadr alohida pickle/IPC orqali yuboriladi; katta bo'lak IPC
xarajatini kamaytiradi.")
    print("\nGPU bo'lsa: ma'lumot parallelligi = DistributedDataParallel (har GPU o'z paketini
ishlaydi, gradientlar all-reduce);")
    print("bu yerda jarayonlar natijani faqat oxirida jamlaydi (reduce), aloqa xarajati ancha
kichik.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Ma'lumot parallelligi (multiprocessing) ===`, `Pool(2) yaratish`

TAJRIBA QILIB KO‘RING

- KADRLAR=60 qiling — qisqa vazifada tezlanish saqlanadimi? Pool xarajati ulushini hisoblang.
- pool.map o'rniga pool.imap_unordered ishlatib, birinchi natija kelgunicha vaqtni (kechikish) o'lchang.
- kadr_ishlov ichidagi takror sonini 6 dan 1 ga tushiring: IPC xarajati hisobga nisbatan qanday o'sadi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Amdahl va Gustafson qonunlari: nazariya va 2 yadroda o'lchov

MAQSAD

S(n)=1/((1−p)+p/n) jadvali p=0,5…0,99 uchun chiqariladi; so'ng «dastur» = Python ketma-ket qism + torch matmul 1 va 2 ipda o'lchanib, S(2) dan parallel ulush p baholanadi va shu p bilan 8 hamda 1024 yadrodagi (GPU) tezlanish bashorat qilinadi. Grafikda Amdahl egri chiziqlari, o'lchangan nuqta va Gustafson chiziqlari ko'rsatiladi. Kutilgan natija: faqat matmul p≈1, ketma-ket qism qo'shilganda p≈0,97 → 1024 yadroda tezlanish atigi ~30×.

> **Kutilgan natija:** Amdahl jadvali (p=0,95: n=inf da 20), o'lchov jadvali S(2) ~1,9–2,0, p baholari, 2 panelli grafik.

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Amdahl va Gustafson qonunlari: nazariy egri chiziqlar + 2 yadroda o'lchangan
parallel ulushni baholash
# S(n) = 1 / ((1-p) + p/n). O'lchangan S(2) dan p = (1 - 1/S) / (1 - 1/n) topiladi va
# shu p bilan 8, 64, 1000 yadrodagi (GPU!) tezlanish chegarasi bashorat qilinadi.
import time
import numpy as np
import torch
import matplotlib.pyplot as plt

torch.manual_seed(0)


def amdahl(p, n):
    return 1.0 / ((1 - p) + p / n)


def gustafson(p, n):
    return (1 - p) + p * n


print("=== Amdahl qonuni ===")
print(f"{'p':>5s} " + " ".join(f"{'n=' + str(n):>8s}" for n in (2, 4, 8, 64, 1024, 'inf')))
for p in (0.5, 0.8, 0.9, 0.95, 0.99):
    satr = [amdahl(p, n) for n in (2, 4, 8, 64, 1024)] + [1 / (1 - p)]
    print(f"{p:5.2f} " + " ".join(f"{s:8.2f}" for s in satr))

# --- O'lchov: "dastur" = ketma-ket qism (Python'da oldindan ishlov) + parallel qism (torch
matmul) ---
def olchov(f, takror=5):
    f(); f()
    v = []
    for _ in range(takror):
        t0 = time.perf_counter(); f(); v.append(time.perf_counter() - t0)
    return sorted(v)[len(v) // 2]

n = 768
A = torch.randn(n, n); B = torch.randn(n, n)

def ketma_ket_qism():
    # parallellashtirilmagan ish: Python siklida kichik amallar (masalan, NMS, log yozish)
```

```
    s = 0.0
    for i in range(60000):
        s += (i % 7) * 0.5
    return s

def dastur(ketma_ket=True):
    if ketma_ket:
        ketma_ket_qism()
    for _ in range(4):
        C = A @ B
    return C

print("\n--- O'lchov: 1 ip vs 2 ip (torch.set_num_threads) ---")
print(f"{'variant':34s} {'1 ip, ms':>9s} {'2 ip, ms':>9s} {'S(2)':>6s} {'p (baho)':>9s}
{'S(8)':>6s} {'S(1024)':>8s}")
natijalar = []
for nom, kk in (("faqat matmul (p ~ 1)", False), ("ketma-ket qism + matmul", True)):
    torch.set_num_threads(1); t1 = olchov(lambda: dastur(kk))
    torch.set_num_threads(2); t2 = olchov(lambda: dastur(kk))
    S2 = t1 / t2
    p = min(1.0, max(0.0, (1 - 1 / S2) / (1 - 1 / 2)))
    natijalar.append((nom, p, S2))
    print(f"{nom:34s} {t1 * 1e3:9.1f} {t2 * 1e3:9.1f} {S2:6.2f} {p:9.3f} {amdahl(p, 8):6.2f}
{amdahl(p, 1024):8.2f}")
torch.set_num_threads(1); t_seq_qism = olchov(ketma_ket_qism)
print(f"Ketma-ket qismning o'zi: {t_seq_qism * 1e3:.1f} ms (o'lchangan) — Amdahl bahosi bilan
solishtiring.")

# --- Grafik ---
nn_ = np.array([1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024])
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
for p in (0.5, 0.9, 0.95, 0.99):
    ax1.plot(nn_, amdahl(p, nn_), label=f"Amdahl p={p}")
for nom, p, S2 in natijalar:
    ax1.plot(nn_, amdahl(p, nn_), "--", label=f"o'lchangan p={p:.2f} ({nom[:14]})")
    ax1.plot([2], [S2], "ko")
ax1.set_xscale("log", base=2); ax1.set_yscale("log", base=2)
ax1.set_xlabel("Protsessorlar soni n"); ax1.set_ylabel("Tezlanish S(n)")
ax1.set_title("Amdahl qonuni: tezlanish chegarasi"); ax1.grid(alpha=0.3, which="both");
ax1.legend(fontsize=7)
for p in (0.5, 0.9, 0.99):
    ax2.plot(nn_, gustafson(p, nn_), label=f"Gustafson p={p}")
    ax2.plot(nn_, amdahl(p, nn_), ":", color="gray")
ax2.set_xscale("log", base=2); ax2.set_yscale("log", base=2)
ax2.set_xlabel("Protsessorlar soni n"); ax2.set_ylabel("Masshtablangan tezlanish")
ax2.set_title("Gustafson (masala hajmi o'sadi) vs Amdahl (nuqtali)"); ax2.grid(alpha=0.3,
which="both"); ax2.legend(fontsize=7)
fig.tight_layout()
print("\nXulosa: GPU (minglab yadro) faqat p -> 1 bo'lganda samarali; shuning uchun paket
(batch) kattalashtiriladi (Gustafson)")
print("va ketma-ket qismlar (NMS, ma'lumot yuklash, CPU-GPU ko'chirish) alohida
optimallashtiriladi.")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Amdahl qonuni ===`, `p (baho)`

TAJRIBA QILIB KO‘RING

- ketma_ket_qism sikl sonini 60 000 dan 600 000 ga oshiring — p va S(1024) qanday tushadi?
- Matritsa o'lchamini 768 dan 1536 ga oshiring (Gustafson: masala hajmi o'sadi) — p qanday o'zgaradi?
- S(2) o'lchovini 5 marta takrorlab p bahosining tarqalishini (min/max) yozing.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**Sof Python vs NumPy: o'lcham egri chizig'i** 1-qadam skriptida n = 16, 32, 64, 128, 192 uchun sof Python va NumPy matmul vaqtlarini o'lchab, log-log grafikda chizing; sof Python uchun vaqt ~n^3 ekanini (log-log og'ish koeffitsiyenti) hisoblab ko'rsating.	oson	5
2	**prange samaradorligi** 2-qadamda n = 128, 256, 512, 1024 uchun prange 1 va 2 ipda tezlanish va parallel samaradorlikni (tezlanish/2) jadvalga yozing; kichik n da samaradorlik past bo'lishining sababini ip yaratish xarajati orqali tushuntiring.	oson	5
3	**Blok o'lchami tadqiqoti** 3-qadamda bs = 8, 16, 32, 64, 128, 256, 512 uchun n=1024 va n=2048 da GFLOP/s o'lchang, grafik chizing va har n uchun eng yaxshi blokni hamda uning uch bloki egallagan KB hajmini L1/L2 kesh o'lchamlari bilan solishtiring.	oson	5
4	**Konvolyutsiya: parallellashtirish o'qi** 4-qadam Numba konvolyutsiyasini uch xil prange o'qi (chiqish kanali, qator, kanal+qator collapse) bilan amalga oshiring; 1 va 2 ipda vaqt, GFLOP/s va tezlanishni jadvalga kiriting, yuk muvozanati va kesh lokalligi nuqtai nazaridan eng yaxshi variantni asoslang.	o'rta	10
5	**im2col xotira va vaqt savdosi** 4-qadam im2col variantida CIN=16/32/64 va 96×96/160×160 kirishlar uchun cols matritsasining hajmini (MB) va vaqtni o'lchang; to'g'ridan-to'g'ri Numba va torch conv2d bilan solishtirib, qaysi hajmdan boshlab im2col xotira cheklovi tufayli foydasiz bo'lishini hisoblab ko'rsating.	o'rta	10
6	**Format va iplar matritsasi** 5-qadamda FP32, BF16, FP16 va INT8 formatlarida n = 256, 512, 1024 va 1, 2 ip uchun GFLOP/s va nisbiy xato jadvalini tuzing; CPU'da past formatlar nima uchun tezlashmasligini va GPU Tensor Core'da (A2 nazariy qismidagi jadval) qanday nisbat kutilishini solishtirib yozing.	o'rta	10
7	**Yadro birlashtirish qo'lda** 5-qadam GELU'sini Numba @njit(parallel=True) bilan bitta o'tishda (fused) yozing; eager torch, F.gelu va Numba fused variantlarini 1 va 2 ipda 8M element uchun o'lchab samarali GB/s ni hisoblang va xotira trafigi nazariy hisobi bilan solishtiring.	o'rta	10
8	**Multiprocessing vs threading vs Numba** 6-qadam kadr_ishlov vazifasini uch usulda parallellashtiring: multiprocessing.Pool(2), concurrent.futures.ThreadPoolExecutor(2) va Numba prange; har biri uchun tezlanish, natija to'g'riligi va xotira sarfini (psutil) o'lchang, GIL ta'sirini threading natijasida ko'rsating va qaysi usul qachon tanlanishini jadval bilan asoslang.	qiyin	15
9	**Amdahl: o'lchangan p va bashorat tekshiruvi** 7-qadam metodikasini 2–4-qadamlardagi barcha parallel variantlarga (prange matmul, prange konvolyutsiya, torch matmul, multiprocessing) qo'llab har biri uchun	qiyin	15

	p ni baholang; ketma-ket qism ulushini qo'lda o'lchab (profil) Amdahl bahosi bilan solishtiring, farq sabablarini (xotira o'tkazuvchanligi, sinxronizatsiya) tushuntiring va jamlovchi grafik chizing.		
10	**CPU Roofline va GPU bashorati** Barcha qadamlardagi o'lchovlarni (sof Python, NumPy, prange, tiled, im2col, torch conv2d, GELU eager/fused) arifmetik intensivlik–unumdorlik tekisligiga joylashtiring: sandbox CPU uchun cho'qqi GFLOP/s ni torch FP32 matmul'ning eng yaxshi natijasidan, xotira o'tkazuvchanligini reduktsiya (2-qadam) natijasidan oling; har nuqta qaysi chegara bilan cheklanganini tushuntiring va A2 matnidagi RTX 3060 ko'rsatkichlari bilan GPU'da kutilgan tezlanishni bashorat qiling.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. EKG monitoring: ko'p kanalli signalni FIR filtrlash va R-tishlarni sanash

oson

Soha: Tibbiyot

Kardiologiya bo'limining markaziy monitoring stansiyasiga 2 000 ta bemor kanalidan 500 Hz chastotali EKG keladi (har kanalda 20 s = 10 000 namuna). Har kanalga 64 koeffitsiyentli FIR polosali filtr qo'llanib, so'ng bo'sag'a bilan R-tishlar sanaladi va yurak urishi hisoblanadi. Tizim CPU serverida ishlaydi va 2 000 kanal 1 s ichida qayta ishlanishi kerak. Siz sof Python'dan boshlab vektorlash, Numba prange, torch conv1d va multiprocessing darajalarida tezlanishni o'lchaysiz. Signal sintetik: sinusoidalar yig'indisi + Gauss shovqini + davriy R-tish impulslari (numpy generator, seed 0).

Yadro	1D FIR filtr (64 tap) + bo'sag'a bilan R-tish sanash (reduktsiya)
O'lchamlar	kanallar 200 / 2 000, namuna 10 000 (float32), taplar 16 / 64
FLOP	2 x 64 x 10 000 x 2 000 = 2,56 GFLOP
Iplar/jarayonlar	Numba 1/2 ip, torch 1/2 ip, Pool 1/2 jarayon
Bo'lak (chunk)	kanallar bo'yicha 1 / 50 / 500
Vaqt chegarasi	2 000 kanal <= 1 s
Ma'lumot	sintetik EKG, seed 0

QADAMLAR

1. (s1) Sof Python FIR (ikki sikl: namuna x tap) 1 kanal x 10 000 namuna uchun vaqt va MFLOP/s; NumPy np.convolve(mode='valid') va 'sirpanuvchi oyna' (np.lib.stride_tricks.sliding_window_view @ h) bilan 200 va 2 000 kanal; tezlanish va maks xato jadvali.
2. (s2) @njit FIR (kanal, namuna, tap sikllari) va @njit(parallel=True) prange kanallar bo'yicha; JIT vaqti; 1 va 2 ipda ms, GFLOP/s, tezlanish; R-tish sanash reduktsiyasini (bo'sag'a oshib o'tish sonini) prange bilan yozib 1 vs 2 ip GB/s samarali o'qishni hisoblang.
3. (s3) Lokallik: tap siklini ichki/tashqi qilib (namuna-tap vs tap-namuna) va kanallarni 50 talik bo'laklarda ishlab (bo'lak L2 keshga sig'adi) GFLOP/s farqini o'lchang; taplar 16 vs 64 da arifmetik intensivlik qanday o'zgaradi.
4. (s4) FIR'ni 4 usulda: Numba 1 ip, Numba prange 2 ip, NumPy sliding_window + GEMM (im2col analogi: 10 000 x 64 matritsa x 64 vektor), torch F.conv1d (2 000 kanal batch, 1 va 2 ip); etalon bilan maks xato < 1e-4; jadval.
5. (s5) torch conv1d 1 vs 2 ip (200/2 000 kanal), float32 vs float64 vs bfloat16 vaqt va nisbiy xato; R-tish sanash eager (3 amal) vs bitta torch.count_nonzero(x[1:] > t) & (x[:-1] <= t) birlashtirilgan ifoda - samarali GB/s.
6. (s6) multiprocessing.Pool(1/2) bemorlar bo'yicha (kanal = vazifa), chunksize 1/50/500; tezlanish, samaradorlik, Pool yaratish xarajati; natija ketma-ket bilan bir xilligini assert qiling.
7. (s7) Amdahl: 'dastur' = Python'da hisobot shakllantirish (ketma-ket, 2 000 kanal uchun sikl) + prange FIR; 1 vs 2 ipda S(2), p bahosi, S(8)/S(1024) bashorati; Gustafson: kanallar 2 000 -> 20

000 (generatsiya bo'laklab) p qanday o'zgaradi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

2-variant. Sun'iy yo'ldosh dala tasvirlarida NDVI va tekstura indekslarini hisoblash o'rta

Soha: Agro

Agro-holding 2048x2048 piksellik 4 kanalli (qizil, yashil, ko'k, NIR) dala tasvirlarini har hafta qayta ishlaydi: NDVI elementlararo hisoblanadi, so'ng 5x5 o'rtacha va dispersiya (tekstura) filtrlari qo'llanib, parsellarga bo'lingan hisobot tuziladi. Bir mavsumda 3 000 tasvir bo'lgani uchun bitta tasvir 0,5 s dan tez ishlanishi kerak. Siz NDVI va 5x5 filtrni sof Python'dan torch conv2d gacha bo'lgan darajalarda parallellashtirib, qaysi daraja xotira, qaysi biri hisob bilan cheklanganini aniqlaysiz. Tasvirlar sintetik: silliq fon + parsel bloklari + shovqin (numpy, seed 1).

Yadro	NDVI = (NIR - R)/(NIR + R) (elementlararo) + 5x5 o'rtacha va dispersiya filtri
O'lchamlar	512 / 1 024 / 2 048 piksel, 4 kanal float32 (2048: 64 MB)
FLOP	5x5 filtr: 2 x 25 x 4,2 M = 210 MFLOP (2048); NDVI ~5 FLOP/piksel
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	satr bo'laklari 32 / 128 / 512 satr
Vaqt chegarasi	1 tasvir <= 0,5 s
Ma'lumot	sintetik 4 kanal tasvir, seed 1

QADAMLAR

1. (s1) NDVI: sof Python ikki sikl (512x512) vs NumPy (512/1024/2048) - ms, M piksel/s, tezlanish; NumPy'da oraliq massivlar (NIR-R, NIR+R, bo'lish) tufayli xotira trafigi MB va samarali GB/s.
2. (s2) @njit 5x5 o'rtacha filtr (to'g'ridan-to'g'ri 4 sikl) va prange satrlar bo'yicha 1/2 ip; JIT vaqti, GFLOP/s, tezlanish; NDVI o'rtachasi va dispersiyasini prange reduktsiya bilan hisoblab 1 vs 2 ip GB/s.
3. (s3) Lokallik: filtrni satr bo'laklarida (32/128/512 satr) ishlab har bo'lak keshda qolishini o'lchang; ustun-birinchi (j tashqi) sikl tartibini qo'shib necha baravar sekinlashishini ko'rsating; 'blok o'lchami ta'siri' jadvali va grafigi.
4. (s4) 5x5 filtrni 4 usulda: Numba 1 ip, prange 2 ip, NumPy 'ajraladigan' filtr (5x1 so'ng 1x5: 2x5 ko'paytma o'rniga 25) yoki im2col + GEMM, torch F.conv2d (1 va 2 ip, 4 kanal batch); dispersiyani E[x^2]-E[x]^2 bilan ikkita conv'dan oling; maks xato < 1e-4.
5. (s5) torch conv2d 1 vs 2 ip (512/1024/2048), FP32 vs BF16 vaqt va xato; NDVI eager (3 amal) vs torch.where bilan bitta ifoda / Numba fused - samarali GB/s 1 va 2 ip.
6. (s6) Pool(1/2) tasvirlar bo'yicha (20 ta 1024x1024 tasvir, har vazifa NDVI + filtr + parsel o'rtachasi), chunksize 1/5/10; tezlanish, Pool xarajati, natija tekshiruvi.
7. (s7) Amdahl: ketma-ket qism = parsel hisobotini Python lug'atida yig'ish, parallel qism = prange filtr; S(2), p, S(8)/S(64)/S(1024); Gustafson: 1024 -> 2048 -> 4096 (4096 uchun faqat 1 kanal, 64 MB) o'lchamlarda p o'sishi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

3-variant. Shahar yo'l tarmog'ida barcha juftliklar orasidagi eng qisqa yo'llar (Floyd-Warshall) qiyin

Soha: Transport

Jamoat transporti boshqarmasi 512 ta bekat/chorrahadan iborat tarmoq uchun barcha juftliklar orasidagi eng qisqa vaqt matritsasini har 10 daqiqada (trafik yangilanganda) qayta hisoblaydi. Floyd-Warshall algoritmi O(n^3) - 512 uchun 134 M yangilanish; sof Python'da bu daqiqalar, Numba'da soniyadan kam. Algoritmdagi k sikli ketma-ket (bog'liqlik bor), i-j sikllari parallellashadi; bloklangan (tiled) Floyd-Warshall esa GPU'dagi umumiy xotira yadrosining aynan o'zi. Siz sof Python'dan prange va bloklashgacha o'tib, k siklining ketma-ketligi Amdahl ulushiga qanday ta'sir qilishini o'lchaysiz. Graf sintetik: tasodifiy geometrik graf, og'irlik = masofa / tezlik (seed 2).

Yadro	Floyd-Warshall: D[i,j] = min(D[i,j], D[i,k] + D[k,j]); keyin 'min-plus' matritsa ko'paytirish varianti
O'lchamlar	n = 64 (Python), 256 / 512 / 1 024 (Numba; 1024 float32 = 4 MB)
Amallar	n^3 yangilanish: 512 -> 134 M, 1024 -> 1,07 G
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2 (manbalar bo'yicha Dijkstra muqobili)
Blok	tiled FW blok 16 / 32 / 64 / 128
Vaqt chegarasi	n=512 <= 2 s
Ma'lumot	tasodifiy geometrik graf, seed 2

QADAMLAR

1. (s1) Sof Python FW n=64 (3 sikl) vs NumPy vektorlangan ichki sikl (har k uchun np.minimum(D, D[:,k,None] + D[None,k,:])) n=64/256/512; ms, M yangilanish/s, tezlanish; NumPy variantida har k da yoziladigan oraliq massiv (n^2 x 4 bayt) trafigi GB/s.
2. (s2) @njit FW (k-i-j) va prange i bo'yicha (k tashqi, ketma-ket) 1/2 ip n=256/512/1024; JIT vaqti, M yangilanish/s, tezlanish; 'o'rtacha yo'l vaqti' va 'erishib bo'lmaydigan juftliklar soni' reduktsiyasini prange bilan hisoblang.
3. (s3) Bloklangan FW (3 fazali: diagonal blok, qator/ustun bloklari, qolganlari) blok 16/32/64/128 bilan 1 ipda; naive k-i-j bilan solishtiring; n=1024 da blok ta'siri jadvali; i-k-j va k-i-j sikl tartibi farqini (D[k,j] qator bo'ylab) o'lchang.
4. (s4) Bir k-qadamni 4 usulda: Numba 1 ip, prange 2 ip, NumPy broadcasting, torch torch.minimum(D, D[:,k:k+1] + D[k:k+1,:]) 1/2 ip; qo'shimcha: min-plus 'matritsa ko'paytirish' (D o D) log2(n) marta - torch'da (D[:,:,None] + D[None,:,:]).min(1) n=256 uchun xotira n^3 x 4 bayt = 64 MB; hammasi bir xil D beradimi (maks xato 0).
5. (s5) torch k-sikli 1 vs 2 ip n=512/1024, float32 vs float16 (masofalar 0-3 600 s oralig'ida xato), eager (qo'shish + minimum alohida) vs torch.minimum(D, torch.add(a, b, out=buf)) buferli variant - samarali GB/s.
6. (s6) Pool(1/2): manbalar bo'yicha bo'lish - har jarayon o'z i-satrlari to'plami uchun Dijkstra (heapq, Python) ni bajaradi (n=512, 64 manba); tezlanish, chunksize 1/8/32, natija FW bilan bir xilligi; nima uchun FW'ni jarayonlarga bo'lish qiyin (har k da sinxronizatsiya) - matn.

7. (s7) Amdahl: FW prange 1 vs 2 ip S(2) dan p; k-sikl boshidagi sinxronizatsiya (n ta bar'yer) xarajatini n=256 vs 1024 da solishtiring; S(8)/S(1024) bashorati; Gustafson: n 2 baravar oshganda ish 8 baravar, bar'yer 2 baravar - p qanday o'sadi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; Bloklangan FW uchun 3 fazaning sxemasi (matn/rasm) va blok o'lchami - kesh hajmi jadvali

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

4-variant. 5G massiv antenna: raqamli nur shakllantirish (beamforming) kompleks matritsa ko'paytirish o'rta

Soha: Telekom

Baza stansiyasining 128 antennali massividan har 1 ms da 20 000 ta kompleks namuna-vektor keladi; har vektorga 128x128 kompleks og'irlik matritsasi (128 nur) qo'llaniladi, so'ng har nur quvvati (|y|^2 yig'indisi) hisoblanadi. Real tizimda bu FPGA/GPU'da bajariladi, lekin siz CPU'da kompleks arifmetikaning parallellashtirish darajalarini o'rganasiz: kompleks ko'paytma 4 haqiqiy ko'paytma + 2 qo'shish, shuning uchun FLOP hisobi 8 x M x N x K. Ma'lumot sintetik: tasodifiy kanal matritsasi + 3 ta 'foydalanuvchi' yo'nalishi (steering vektorlar), seed 3.

Yadro	kompleks GEMM Y = X W (X: 20 000 x 128, W: 128 x 128, complex64) + nur quvvati reduktsiyasi
O'lchamlar	namuna 2 000 / 20 000, antenna 64 / 128, nur 128
FLOP	8 x 20 000 x 128 x 128 = 2,6 GFLOP
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	namuna bloklari 64 / 256 / 1 024
Vaqt chegarasi	20 000 namuna <= 100 ms (10 x real vaqt)
Ma'lumot	sintetik kanal + 3 steering vektor, seed 3

QADAMLAR

1. (s1) Sof Python kompleks matmul (Python complex, 3 sikl) 200 x 64 x 64 uchun vs NumPy complex64 @ (2 000/20 000 namuna, 64/128 antenna); ms, GFLOP/s (8MNK), tezlanish, maks xato; complex64 vs complex128 BLAS tezligi.
2. (s2) @njit kompleks matmul (haqiqiy/mavhum qismlarni alohida float32 massivlarda saqlab 4 ko'paytma) va prange namunalar bo'yicha 1/2 ip; JIT vaqti, GFLOP/s, tezlanish; nur quvvati sum|y|^2 prange reduktsiyasi 1 vs 2 ip GB/s.
3. (s3) Namunalarni 64/256/1024 lik bloklarda ishlab (W 128x128x8 bayt = 128 KB keshda qoladi) GFLOP/s; W'ni transpozitsiya qilib (nur bo'ylab ketma-ket) qaysi tartib vektorlashishini o'lchang; blok ta'siri jadvali.
4. (s4) Beamforming'ni 4 usulda: Numba 1 ip, prange 2 ip, NumPy complex64 GEMM, torch complex64 matmul 1/2 ip (yoki haqiqiy 2x2 blok ko'rinishida [Xr -Xi; Xi Xr] float32 GEMM - 4 real GEMM); etalon bilan maks xato; jadval.
5. (s5) torch matmul 1 vs 2 ip (2 000/20 000), complex64 vs complex128 vs 'real 2x2 blok float32' vs bfloat16 blok; nur quvvati eager (abs, kvadrat, yig'indi) vs torch.sum(y.real*y.real + y.imag*y.imag) vs Numba fused - GB/s.
6. (s6) Pool(1/2) 1 ms 'slotlar' bo'yicha (40 slot x 20 000 namuna generatsiya qilinadi), chunksize 1/5/20; tezlanish, IPC xarajati (20 000 x 128 x 8 bayt = 20 MB har slot!) va nima uchun natijani emas, faqat quvvat vektorini qaytarish kerak.
7. (s7) Amdahl: ketma-ket qism = steering vektorlarini Python'da hisoblash + nur tanlash (argmax, log), parallel = prange GEMM; S(2), p, S(8)/S(1024); Gustafson: namuna 20 000 -> 200 000

(bo'laklab) p o'zgarishi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

5-variant. Transformator radiatorida issiqlik tarqalishini 2D stensil (Jacobi) bilan modellashtirish o'rta

Soha: Energetika

Elektr tarmog'i kompaniyasi kuch transformatori radiatorining termal holatini real vaqtda baholash uchun 2D issiqlik o'tkazuvchanlik tenglamasini 1024x1024 to'rda Jacobi iteratsiyasi bilan yechadi (har iteratsiya 5 nuqtali stensil). Har 1 s da yangi chegara sharoiti (yuk toki) keladi va 200 iteratsiya 0,5 s ichida bajarilishi kerak. Stensil - xotira bilan cheklangan klassik yadro (AI ~0,5 FLOP/bayt), shuning uchun 2 ip tezlanishi hisob emas, xotira o'tkazuvchanligi bilan belgilanadi; vaqt bo'yicha bloklash (temporal tiling) va float32 bu chegarani yengillashtiradi. Boshlang'ich maydon va issiqlik manbalari sintetik (seed 4).

Yadro	5 nuqtali Jacobi stensil: u_new = 0,25 (u[i-1,j] + u[i+1,j] + u[i,j-1] + u[i,j+1]) + f; qoldiq normasi reduktsiyasi
O'lchamlar	to'r 256 / 512 / 1 024 / 2 048 (2048 float64 = 32 MB x 2)
FLOP	6 FLOP/nuqta; 1024^2 x 200 iter = 1,26 GFLOP
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2 (domen dekompozitsiyasi)
Blok	satr bo'laklari 16 / 64 / 256; vaqt bloki 1 / 2 / 4 iteratsiya
Vaqt chegarasi	200 iter @1024 <= 0,5 s
Ma'lumot	sintetik boshlang'ich maydon + 3 manba, seed 4

QADAMLAR

1. (s1) Sof Python Jacobi 128x128 x 10 iter vs NumPy slicing (u[1:-1,2:] + ...) 256/512/1024 x 200 iter; ms, M nuqta/s, GFLOP/s, tezlanish; NumPy variantida har iteratsiyada 4 ta oraliq massiv yozilishi - samarali GB/s.
2. (s2) @njit Jacobi (ikki sikl, ikki bufer almashinadi) va prange satrlar bo'yicha 1/2 ip; JIT vaqti; GFLOP/s va GB/s (o'qish 5 + yozish 1 qiymat/nuqta); qoldiq normasini (sum (u_new-u)^2) prange reduktsiya bilan har 10 iteratsiyada hisoblang - 1 vs 2 ip.
3. (s3) Lokallik: ustun-birinchi tartib (j tashqi) vs satr-birinchi; satr bo'laklari 16/64/256 (bo'lak x 3 satr keshda); vaqt bo'yicha bloklash: 2 va 4 iteratsiyani bitta o'tishda (ikkita qo'shimcha halo satr bilan) bajarib GB/s ga nisbatan GFLOP/s o'sishini ko'rsating; float32 vs float64.
4. (s4) Stensilni 4 usulda: Numba 1 ip, prange 2 ip, NumPy slicing, torch F.conv2d 3x3 yadro [[0,.25,0],[.25,0,.25],[0,.25,0]] 1/2 ip (padding bilan chegara sharti); 200 iteratsiyadan keyin maydonlar maks farqi < 1e-6 (float64) yoki < 1e-3 (float32).
5. (s5) torch conv2d stensil 1 vs 2 ip (512/1024/2048), float64 vs float32 vs bfloat16 (200 iter yig'ilgan xato!); qoldiq normasi eager (ayirma, kvadrat, yig'indi) vs torch.sum((a-b)**2) bitta ifoda vs Numba fused - GB/s.
6. (s6) Pool(1/2) domen dekompozitsiyasi: to'rni 2 ta gorizontal yarmiga bo'lib, har jarayon o'z yarmini 200 iteratsiya ishlaydi va har iteratsiyada halo satrni almashadi (mp.Pipe) - tezlanish va aloqa xarajati; muqobil: mustaqil 8 ta ssenariy (turli yuk toki) ni Pool'da bo'lish - qaysi biri samarali va nima uchun.

7. (s7) Amdahl: ketma-ket qism = chegara sharti va manbalarni Python'da yangilash (har iteratsiya), parallel = prange stensil; S(2), p, S(8)/S(1024); Gustafson: to'r 512 -> 2048 da p o'sishi; grafik; xulosada xotira chegarasi tufayli S(2) < 2 ekanini GB/s bilan asoslang.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

6-variant. Monte-Karlo bilan opsion narxlash va portfel xavfini (VaR) baholash

oson

Soha: Moliya

Investitsiya kompaniyasining xavf bo'limi har kuni 200 000 ta narx traektoriyasi (geometrik Broun harakati, 252 kunlik qadam) bilan Yevropa va Osiyo opsionlarini narxlaydi hamda portfel VaR'ini hisoblaydi. Hisob tungi oynada 5 daqiqa ichida tugashi kerak, lekin treyderlar 'nima bo'lsa' tahlilini kunduzi 2 s ichida kutishadi. Monte-Karlo - 'uyatli parallel' (embarrassingly parallel) masala: traektoriyalar mustaqil, faqat oxirida reduktsiya; siz bu p -> 1 holatda Amdahl va Gustafson qonunlari qanday ko'rinishini o'lchaysiz. Tasodifiy sonlar: numpy default_rng (seed 5), Numba'da har ip o'z seed'i bilan.

Yadro	GBM traektoriya: S_t+1 = S_t exp((r - s^2/2) dt + s sqrt(dt) Z) + to'lov funksiyasi + o'rtacha/kvantil reduktsiyasi
O'lchamlar	traektoriya 20 000 / 200 000, qadam 252, aktivlar 1 / 5 (korrelyatsiyalangan, Cholesky)
Amallar	200 000 x 252 = 50,4 M normal son + ~10 FLOP har biri
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Bo'lak	traektoriya bo'laklari 1 000 / 10 000 / 50 000
Vaqt chegarasi	200 000 traektoriya <= 2 s
Ma'lumot	S0=100, r=0,05, sigma=0,2, T=1; 5 aktiv korrelyatsiya matritsasi, seed 5

QADAMLAR

1. (s1) Sof Python (random.gauss, ikki sikl) 2 000 traektoriya vs NumPy vektorlangan (barcha traektoriyalar bir yo'la, 252 qadam sikli) 20 000/200 000; ms, M qadam/s, tezlanish; Yevropa call narxi Black-Scholes formulasi bilan solishtirilsin (xato < 1 %); NumPy'da 200 000 x 252 float64 = 403 MB oraliq massiv - xotira muammosini bo'laklab yechish.
2. (s2) @njit traektoriya sikli (traektoriya bo'yicha tashqi, qadam ichki, np.random.normal njit ichida) va prange traektoriyalar bo'yicha 1/2 ip; JIT vaqti; M qadam/s, tezlanish; to'lov o'rtachasi va dispersiyasi prange reduktsiya; 2 ipda natija 1 ipdagidan nima uchun farq qiladi (tasodifiy oqim) va buni qanday nazorat qilish.
3. (s3) Lokallik: 'qadam tashqi, traektoriya ichki' (NumPy uslubi, 200 000 x 8 bayt vektor har qadamda) vs 'traektoriya tashqi, qadam ichki' (Numba, 1 skalyar) tartiblarini o'lchang; bo'lak 1 000/10 000/50 000 traektoriya (bo'lak x 8 bayt L2 ga sig'adimi) ta'siri jadvali; Osiyo opsioni uchun yo'l o'rtachasini saqlash xotirani qanday oshiradi.
4. (s4) 5 aktivli korrelyatsiyalangan traektoriyalar: Z (5 x N) ni Cholesky L bilan ko'paytirish (kichik GEMM 5x5 x 5xN) 4 usulda - Numba 1 ip, prange 2 ip, NumPy L @ Z, torch 1/2 ip; har qadamda GEMM vs barcha qadamlarni (5 x 252N) bitta GEMM'da; etalon bilan maks xato.
5. (s5) torch: traektoriya generatsiyasi torch.randn + exp cumsum 1 vs 2 ip, float32 vs float64 (narx xatosi), eager (5 alohida amal) vs oldindan hisoblangan drift bilan bitta exp(cumsum(...)) - samarali GB/s; 200 000 x 252 float32 = 200 MB.

6. (s6) Pool(1/2) seedlar bo'yicha (har vazifa = 10 000 traektoriya o'z seed'i bilan, natija = yig'indi va kvadratlar yig'indisi), chunksize 1/2/5; tezlanish, Pool xarajati; natijani qaytarishda 10 000 x 8 bayt emas, 2 ta son yuborish IPC'ni qanday kamaytiradi.
7. (s7) Amdahl: faqat prange simulyatsiya (p ~ 1) va 'simulyatsiya + Python'da hisobot/kvantil (np.percentile emas, sorted) ' uchun S(2), p, S(8)/S(1024); Gustafson: 20 000 -> 200 000 -> 2 000 000 (bo'laklab) traektoriyada p -> 1; grafik; GPU'da (RTX 3060, 3 584 yadro) kutilgan tezlanishni bashorat qiling.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

7-variant. Milliy test natijalari: savollar statistikasi va korrelyatsiya matritsasi (X^T X) oson

Soha: Ta'lim

Test markazi 50 000 abituriyentning 200 ta savolga javob matritsasini (1 - to'g'ri, 0 - noto'g'ri) tahlil qiladi: har abituriyent bali (satr yig'indisi), har savol qiyinligi (ustun o'rtachasi), savollar orasidagi korrelyatsiya (200x200, X^T X orqali) va har savolning umumiy ball bilan nuqta-biserial korrelyatsiyasi. Natija imtihondan keyin 10 daqiqa ichida, 'nima bo'lsa' tahlili uchun 1 s ichida kerak. Siz reduktsiya va Gram matritsasi (X^T X) uchun sof Python'dan BLAS va prange gacha darajalarni o'lchaysiz; bu yerda X^T X - tor (200 ustun) matritsa ko'paytirish, AI balandligi N bilan o'sadi. Javoblar sintetik: 2 parametrli IRT modeli (qobiliyat, qiyinlik) bo'yicha Bernoulli (seed 6).

Yadro	satr/ustun yig'indilari (reduktsiya) + Gram matritsasi X^T X (50 000 x 200) + korrelyatsiya normalizatsiyasi
O'lchamlar	abituriyent 5 000 / 50 000, savol 50 / 200; X float32 (40 MB) va int8 (10 MB)
FLOP	2 x 50 000 x 200 x 200 = 4 GFLOP
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	abituriyent bloklari 500 / 5 000 / 50 000 (qisman Gram yig'indisi)
Vaqt chegarasi	to'liq tahlil <= 1 s
Ma'lumot	IRT 2PL sintetik javoblar, seed 6

QADAMLAR

1. (s1) Sof Python: ballar (satr yig'indisi) va Gram matritsasi 500 x 50 uchun uch sikl vs NumPy X.sum(1), X.T @ X 5 000/50 000 x 50/200 (float32 va float64); ms, GFLOP/s, tezlanish, maks xato; int8 matritsani float32 ga o'tkazish vaqtini alohida o'lchang.
2. (s2) @njit Gram (i-k-j tartib, simmetriyadan foydalanib yarmini hisoblash) va prange abituriyentlar bloklari bo'yicha (har ip o'z qisman G'ini yig'adi, so'ng qo'shiladi) 1/2 ip; JIT vaqti, GFLOP/s, tezlanish; ustun o'rtachasi (qiyinlik) prange reduktsiya 1 vs 2 ip GB/s.
3. (s3) Bloklash: X'ni 500/5 000/50 000 satrli bloklarda o'qib qisman Gram'larni yig'ish (blok x 200 x 4 bayt L2 ga sig'adimi); X satr-ustun (C) vs ustun-satr (Fortran) joylashuvida X.T @ X tezligi; blok ta'siri jadvali.
4. (s4) Nuqta-biserial korrelyatsiyani 4 usulda: Numba 1 ip (savollar bo'yicha sikl), prange 2 ip, NumPy ((X - p) * (score - m)[:,None]).mean(0) / std, torch 1/2 ip (bitta matvec X.T @ score_centered); maks xato < 1e-5; jadval.
5. (s5) torch X.T @ X 1 vs 2 ip (5 000/50 000), float32 vs bfloat16 vs int8 (torch._int_mm, 0/1 uchun aniq!) - vaqt va xato; korrelyatsiya normalizatsiyasi eager (4 amal) vs bitta ifoda - GB/s.
6. (s6) Pool(1/2) hududlar bo'yicha (14 ta hudud, har biri 3 500-4 000 abituriyent, har vazifa qisman yig'indilar + qisman Gram qaytaradi), chunksize 1/2/7; tezlanish, natija to'liq hisob bilan bir xilligi, IPC hajmi (200x200 float64 = 320 KB har vazifa).

7. (s7) Amdahl: ketma-ket qism = hisobot (200 savol uchun Python'da matn satrlari, saralash), parallel = prange Gram; S(2), p, S(8)/S(1024); Gustafson: abituriyent 5 000 -> 50 000 -> 500 000 (bloklab generatsiya) da p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

8-variant. Turbina vibratsiya monitoringi: ko'p kanalli FFT, Welch spektri va chastota cho'qqilari o'rta

Soha: Sanoat

Issiqlik elektr stansiyasining 2 048 ta vibratsiya sensori 25,6 kHz chastotada yoziladi; har 320 ms da (8 192 namuna) har kanal uchun Hann oynali FFT, Welch quvvat spektri (8 segment x 1 024) va 3 ta asosiy cho'qqi topiladi; natija 320 ms oynaning o'zida tayyor bo'lishi kerak (real vaqt). FFT O(N log N) - matmul'dan past AI, shuning uchun iplar tezlanishi va formatlar (float32/complex64) ta'siri boshqacha; oyna ko'paytirish va |X|^2 esa elementlararo amallar bo'lib birlashtirishga (fusion) nomzod. Signal sintetik: 1x, 2x aylanish garmonikalari + podshipnik chastotasi + shovqin (seed 7).

Yadro	Hann oyna x signal -> FFT -> |X|^2 -> Welch o'rtacha -> argmax cho'qqilar
O'lchamlar	kanal 256 / 2 048, namuna 8 192 (float32: 64 MB), segment 1 024
FLOP	~5 N log2 N = 0,53 MFLOP/kanal FFT; 2 048 kanal -> 1,1 GFLOP + Welch 8 segment
Iplar/jarayonlar	numpy.fft (1 ip), torch.fft 1/2 ip, Numba 1/2 ip (oyna, |X|^2, Welch), Pool 1/2
Bo'lak	kanal bloklari 16 / 128 / 1 024
Vaqt chegarasi	2 048 kanal <= 320 ms
Ma'lumot	sintetik vibratsiya, seed 7

QADAMLAR

1. (s1) Sof Python DFT (ikki sikl, N=512, 1 kanal) vs np.fft.rfft (N=512 va 8 192, 256/2 048 kanal, axis=1); ms, MFLOP/s (5 N log N formulasi), tezlanish, maks xato; float32 vs float64 kirishda rfft vaqti.
2. (s2) @njit Hann oyna + |X|^2 + Welch o'rtachasi (FFT'ni np.fft bilan qoldirib, elementlararo qismni Numba'da) va prange kanallar bo'yicha 1/2 ip; JIT vaqti; GB/s samarali; umumiy quvvat (sum |X|^2) prange reduktsiya 1 vs 2 ip; FFT ulushi vs elementlararo ulush profil jadvali.
3. (s3) Lokallik: kanallarni 16/128/1 024 lik bloklarda (blok x 8 192 x 4 bayt) ishlab rfft + Welch GFLOP/s; signal massivi (kanal, namuna) vs (namuna, kanal) joylashuvi (axis=0 FFT) farqi; blok ta'siri jadvali.
4. (s4) Welch spektrini 4 usulda: Numba 1 ip (FFT'siz: 1 024 nuqtali DFT'ni 1 024x1 024 kosinus/sinus matritsa bilan GEMM - 'DFT as GEMM'), prange 2 ip, NumPy rfft + reshape, torch.fft.rfft 1/2 ip (2 048 x 8 segment batch); maks xato < 1e-3 (float32); DFT-GEMM FLOP'i FFT'dan necha baravar ko'p, lekin GFLOP/s yuqori - tushuntiring.
5. (s5) torch.fft.rfft 1 vs 2 ip (256/2 048 kanal), float32 vs float64 (complex64/128); oyna x signal + |X|^2 eager (3 amal) vs torch.abs(rfft(x*w))**2 ... vs Numba fused oyna - samarali GB/s; cho'qqi topish torch.topk 3 vs Numba.
6. (s6) Pool(1/2) sensor guruhlari bo'yicha (16 guruh x 128 kanal, har vazifa 3 cho'qqi chastotasini qaytaradi), chunksize 1/2/8; tezlanish, IPC (128 x 8 192 x 4 = 4 MB har vazifa) ulushi; natija tekshiruvi.

7. (s7) Amdahl: ketma-ket qism = ogohlantirish qoidalari (Python'da 2 048 kanal sikli, cho'qqi chegara bilan solishtirish), parallel = torch rfft 2 ip; S(2), p, S(8)/S(1024); Gustafson: namuna 8 192 -> 65 536 (kanal 256) da p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

9-variant. Video-kuzatuv: kadrlar farqi orqali harakatni aniqlash va zona statistikasi oson

Soha: Xavfsizlik

Ob'ekt qo'riqlash xizmatining 16 ta kamerasi (240x320, 10 kadr/s) uchun oddiy, lekin keng tarqalgan 'harakat detektori' ishlaydi: ketma-ket kadrlar farqi, 5x5 quti filtri (box filter), bo'sag'a va 4x4 zonalar bo'yicha harakat piksellari ulushi. 16 kamera x 10 kadr/s = 160 kadr/s real vaqtda bajarilishi kerak, ya'ni bir kadrga 6 ms. Hamma amallar elementlararo yoki kichik filtr - ya'ni xotira bilan cheklangan; siz vektorlash, birlashtirish (fusion), prange va jarayonlar bu turdagi yadroga qancha foyda berishini o'lchaysiz. Kadrlar sintetik: statik fon + harakatlanuvchi to'rtburchak + shovqin (seed 8); 1 kamera x 300 kadr xotirada (23 MB), qolganlari jarayon ichida hosil qilinadi.

Yadro	|F_t - F_t-1| -> 5x5 quti filtr -> bo'sag'a -> 4x4 zona yig'indisi (reduktsiya)
O'lchamlar	kadr 240x320 uint8 (float32 ishlovda), kadrlar 300 / 1 kamera; 16 kamera Pool'da
Amallar	~60 FLOP/piksel (filtr 25 + farq + bo'sag'a) x 76 800 = 4,6 MFLOP/kadr
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2 (kameralar bo'yicha)
Blok	kadr paketlari 1 / 10 / 50 kadr
Vaqt chegarasi	<= 6 ms/kadr (160 kadr/s)
Ma'lumot	sintetik kadrlar, seed 8

QADAMLAR

1. (s1) Sof Python (ikki sikl, farq + bo'sag'a + zona yig'indisi, 5x5 filtrsiz) 10 kadr vs NumPy (farq, scipy.ndimage.uniform_filter 5x5 yoki kumulyativ yig'indi bilan quti filtr, reshape bilan zona yig'indisi) 300 kadr; ms/kadr, M piksel/s, tezlanish; NumPy'dagi oraliq massivlar soni va samarali GB/s.
2. (s2) @njit to'liq zanjir (bitta o'tishda farq + 5x5 yig'indi + bo'sag'a + zona hisoblagich) va prange satrlar bo'yicha 1/2 ip; JIT vaqti; ms/kadr, GB/s; zona hisoblagichlar reduktsiyasi (16 ta hisoblagich prange'da - poyga holati!) ni to'g'ri yozing (har ip o'z massivini yig'ib keyin qo'shadi).
3. (s3) Lokallik: 5x5 quti filtrni to'g'ridan-to'g'ri (25 o'qish) vs ajraladigan (5 gorizontal + 5 vertikal) vs integral tasvir (4 o'qish) bilan o'lchang; kadrlarni 1/10/50 talik paketlarda (paket x 300 KB keshga sig'adimi) ishlab GB/s; jadval.
4. (s4) Quti filtrni 4 usulda: Numba 1 ip, prange 2 ip, NumPy kumulyativ yig'indi (integral), torch F.avg_pool2d(stride 1, padding 2) yoki conv2d 5x5 (1/2 ip, 10 kadr batch); maks xato < 1e-4; jadval.
5. (s5) torch zanjir 1 vs 2 ip (10/50 kadr batch), uint8 -> float32 vs float16 (CPU) vaqt; eager (farq, abs, filtr, bo'sag'a, yig'indi alohida) vs iloji boricha birlashtirilgan ifoda vs Numba fused - samarali GB/s 1 va 2 ip.
6. (s6) Pool(1/2) kameralar bo'yicha (16 kamera x 300 kadr, har jarayon o'z kadrlarini seed bilan hosil qiladi va 4x4 zona statistikasini qaytaradi), chunksize 1/2/8; tezlanish, samaradorlik, Pool

xarajati; nima uchun kadrlarni emas, seed'ni yuborish kerak (IPC).

7. (s7) Amdahl: ketma-ket qism = ogohlantirish mantiqi va log (Python, har kadr), parallel = prange zanjir; S(2), p, S(8)/S(1024); Gustafson: kadr 240x320 -> 480x640 -> 960x1280 da p; grafik; xulosada 6 ms/kadr talabini qaysi usul qaysi ip sonida bajarishini jadvalda ko'rsating.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

10-variant. Tavsiya tizimi: 20 000 mahsulot embedding'lari orasida bloklangan kosinus o'xshashlik va top-k qiyin

Soha: Chakana savdo

Chakana savdo tarmog'ining onlayn do'koni 20 000 mahsulot uchun 128 o'lchamli embedding'lar asosida 'o'xshash mahsulotlar' (har mahsulot uchun top-10) ro'yxatini har kecha yangilaydi. To'liq o'xshashlik matritsasi 20 000 x 20 000 x 4 bayt = 1,6 GB - 1,5 GB xotiraga sig'maydi, shuning uchun hisob bloklanadi: 2 000 qatorli bloklar (160 MB) hisoblanib, darhol top-10 ajratiladi va blok tashlanadi. Bu GEMM (102 GFLOP) + argpartition (xotira bilan cheklangan) kombinatsiyasi; siz blok o'lchami, iplar va jarayonlarning jami vaqtga ta'sirini hamda Amdahl'da argpartition ulushini o'lchaysiz. Embedding'lar sintetik: 50 ta klaster markazi + shovqin, normallashtirilgan (seed 9).

Yadro	S = E E^T (bloklab) -> har qator top-k (argpartition) -> indekslar; normalizatsiya (reduktsiya)
O'lchamlar	mahsulot 2 000 / 20 000, o'lcham d = 64 / 128, k = 10
FLOP	2 x 20 000^2 x 128 = 102 GFLOP; S bloki 2 000 x 20 000 = 160 MB
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	qator bloki 250 / 1 000 / 2 000 / 5 000
Xotira chegarasi	<= 600 MB (1,5 GB muhit)
Ma'lumot	50 klaster + shovqin, seed 9

QADAMLAR

1. (s1) Sof Python kosinus o'xshashlik (3 sikl) 200 x 200 x 64 vs NumPy E @ E.T 2 000 x 2 000 (d 64/128); ms, GFLOP/s, tezlanish; top-10: Python sorted vs np.argpartition - alohida vaqt; 20 000 uchun to'liq matritsa xotirasini hisoblab, nima uchun bloklash shart ekanini ko'rsating.
2. (s2) @njit bloksiz skalyar ko'paytma + top-k (har qator uchun 'insertion' bilan 10 ta eng kattani saqlash, S saqlanmaydi!) va prange qatorlar bo'yicha 1/2 ip 2 000/20 000 (20 000 uchun vaqt chegarasi - 5 000 qatorda o'lchab ekstrapolyatsiya); JIT vaqti, GFLOP/s; normalizatsiya (satr normasi) prange reduktsiya 1 vs 2 ip.
3. (s3) Bloklash: blok 250/1 000/2 000/5 000 qator uchun GEMM (BLAS) + argpartition vaqtini alohida o'lchang, blok xotirasini (blok x 20 000 x 4 bayt) va psutil bilan haqiqiy RSS cho'qqisini yozing; eng yaxshi blok va uning sababi (GEMM samaradorligi vs kesh/xotira); jadval + grafik.
4. (s4) Bitta blokni (2 000 x 20 000) 4 usulda: Numba 1 ip (ikj), prange 2 ip, NumPy BLAS, torch matmul 1/2 ip; top-k uchun torch.topk vs np.argpartition; etalon bilan indekslar mosligi (to'plam tengligi, bir xil qiymatlar uchun tartib farqini hisobga oling).
5. (s5) torch matmul blok 1 vs 2 ip, float32 vs bfloat16 vs int8 (torch._int_mm, embedding'larni 127 ga masshtablab) - vaqt, top-10 ro'yxatlar mosligi (%); normalizatsiya eager (kvadrat, yig'indi, sqrt, bo'lish) vs F.normalize - GB/s.
6. (s6) Pool(1/2) qator bloklari bo'yicha (10 blok x 2 000, E fork orqali meros bo'ladi - nusxalanmaydi, natija 2 000 x 10 indeks), chunksize 1/2/5; tezlanish; 2 jarayon x BLAS 2 ip = 4

ip 2 yadroda - ortiqcha obuna (oversubscription) ta'sirini OMP_NUM_THREADS=1 bilan solishtiring.

7. (s7) Amdahl: GEMM (parallel, BLAS 2 ip) + argpartition (NumPy, 1 ip - ketma-ket) ulushlari: profil bilan p ni to'g'ridan-to'g'ri hisoblang va S(2) o'lchovidan olingan p bilan solishtiring; S(8)/S(1024) - GPU'da argpartition ham parallel bo'lsa (torch.topk) p qanday o'zgaradi; Gustafson: 20 000 -> 200 000 mahsulot (ekstrapolyatsiya) ; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; Xotira profili: blok o'lchami vs RSS cho'qqisi (psutil) grafigi

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

11-variant. Yetkazib berish nuqtalari orasida masofa matritsasi (haversine) va eng yaqin ombor oson

Soha: Logistika

Kuryerlik kompaniyasi shahar bo'ylab 4 000 ta yetkazib berish nuqtasi va 40 ta ombor koordinatalariga ega. Har kuni ertalab barcha nuqtalar orasidagi masofa matritsasi (haversine formulasi, 16 M juft) va har nuqta uchun eng yaqin ombor hamda 5 ta eng yaqin qo'shni hisoblanadi (marshrutlash uchun). Haversine - trigonometrik funksiyalarga boy elementlararo yadro (AI yuqori, ~25 FLOP/juft), shuning uchun vektorlash va prange yaxshi tezlanish beradi, lekin NumPy broadcasting 16 M x 8 bayt oraliq massivlar yaratadi. Koordinatalar sintetik: 6 ta tuman markazi atrofida normal taqsimot (seed 10).

Yadro	haversine(lat1, lon1, lat2, lon2) barcha juftliklar + argmin (eng yaqin ombor) + top-5 qo'shni
O'lchamlar	nuqta 500 / 4 000, ombor 40; matritsa 4 000^2 float32 = 64 MB
FLOP	~25 FLOP/juft x 16 M = 0,4 GFLOP (trig bilan)
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	qator bloklari 100 / 500 / 4 000
Vaqt chegarasi	4 000 nuqta <= 1 s
Ma'lumot	6 tuman markazi atrofida sintetik koordinatalar, seed 10

QADAMLAR

1. (s1) Sof Python haversine (math.sin/cos, ikki sikl) 500 x 500 vs NumPy broadcasting (lat[:,None], lon[None,:]) 500/4 000 float32 va float64; ms, M juft/s, tezlanish, maks xato (metrda); NumPy'da yaratiladigan oraliq massivlar soni va MB.
2. (s2) @njit haversine (ikki sikl, oldindan hisoblangan sin/cos lat) va prange qatorlar bo'yicha 1/2 ip; JIT vaqti; M juft/s, tezlanish; eng yaqin ombor (argmin 40 ta) va o'rtacha masofa prange reduktsiya 1 vs 2 ip.
3. (s3) Lokallik: qator bloklari 100/500/4 000 (blok x 4 000 x 4 bayt) bilan masofa + top-5 ni blokda hisoblab matritsani saqlamaslik; sin/cos ni oldindan hisoblash (4 000 x 4 qiymat) FLOP'ni necha baravar kamaytiradi; simmetriyadan foydalanib yarmini hisoblash; jadval.
4. (s4) Masofani 4 usulda: Numba 1 ip, prange 2 ip, NumPy broadcasting, torch (torch.sin/cos broadcasting yoki torch.cdist 3D dekart koordinatalarida - yer sharida xorda masofasi, 2 asin(c/2R) bilan haversine'ga aylantiring) 1/2 ip; maks xato < 1 m; jadval.
5. (s5) torch 1 vs 2 ip (500/4 000), float32 vs float64 vs float16 (xato metrda!), eager (8 alohida trig/arifmetik amal) vs iloji boricha kam oraliq massiv (in-place out=) vs Numba fused - GB/s.
6. (s6) Pool(1/2) tumanlar bo'yicha (6 tuman, har vazifa o'z nuqtalari uchun eng yaqin ombor va top-5 ni qaytaradi; nuqtalar fork orqali meros), chunksize 1/2/3; tezlanish, yuk muvozanati (tumanlar har xil kattalikda!) - imap_unordered bilan solishtiring.
7. (s7) Amdahl: ketma-ket qism = marshrut hisobotini Python'da shakllantirish (4 000 satr), parallel = prange masofa; S(2), p, S(8)/S(1024); Gustafson: 500 -> 4 000 -> 12 000 nuqta

(bloklab) p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

12-variant. Havo sifati xaritasi: stansiya o'lchovlarini IDW bilan shahar to'riga interpolyatsiya qilish oson

Soha: Shahar

Shahar ekologiya portali har 10 daqiqada 200 ta stansiyaning PM2.5 o'lchovlaridan 1 000 x 1 000 nuqtali xarita (teskari masofa og'irligi, IDW, p=2) tuzadi: har to'r nuqtasi uchun 200 stansiyagacha masofa, og'irlik 1/d^2 va og'irlikli o'rtacha. Jami 200 M masofa hisobi; xarita 5 s ichida tayyor bo'lishi kerak. Yadro - to'r nuqtalari bo'yicha 'uyatli parallel', ichki sikl esa reduktsiya (og'irliklar yig'indisi); NumPy broadcasting to'liq 1 000 000 x 200 x 4 bayt = 800 MB matritsa yaratadi - bo'laklash shart. Stansiyalar va qiymatlar sintetik: 3 ta ifloslanish manbai atrofida (seed 11).

Yadro	IDW: z(x) = sum(w_i z_i)/sum(w_i), w_i = 1/d_i^2 (200 stansiya) har to'r nuqtasi uchun
O'lchamlar	to'r 200x200 / 1 000x1 000, stansiya 50 / 200
FLOP	~8 FLOP x 1 M nuqta x 200 stansiya = 1,6 GFLOP
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Bo'lak	to'r satr bo'laklari 10 / 50 / 200 satr (bo'lak x 1 000 x 200 x 4 bayt)
Vaqt chegarasi	1 000x1 000 xarita <= 5 s
Ma'lumot	200 stansiya, 3 manba, seed 11

QADAMLAR

1. (s1) Sof Python IDW (3 sikl) 200x200 to'r x 50 stansiya vs NumPy broadcasting (to'r bo'laklab 50 satr) 200x200 va 1 000x1 000 x 200; ms, M nuqta-stansiya/s, tezlanish, maks xato; bo'laksiz broadcasting xotirasini hisoblab ko'rsating.
2. (s2) @njit IDW (to'r i, j tashqi, stansiya ichki - 2 ta akkumulyator) va prange satrlar bo'yicha 1/2 ip; JIT vaqti; GFLOP/s, tezlanish; xarita maksimumi va o'rtachasi prange reduktsiya 1 vs 2 ip.
3. (s3) Lokallik: stansiya siklini tashqi (200 ta to'liq xarita o'tishi) vs ichki (1 o'tish) qilib GB/s farqini o'lchang; satr bo'laklari 10/50/200 (NumPy variantida) ta'siri; stansiya koordinatalarini float32 massivda ketma-ket saqlash (SoA) vs (200, 2) (AoS); jadval.
4. (s4) IDW'ni 4 usulda: Numba 1 ip, prange 2 ip, NumPy bo'lakli broadcasting, torch (torch.cdist to'r-stansiya + og'irlik matvec: W @ z / W.sum(1)) 1/2 ip; maks xato < 1e-4; torch variantida cdist + 2 matvec 'GEMM'ga keltirish' g'oyasi.
5. (s5) torch 1 vs 2 ip (200x200 / 1 000x1 000), float32 vs float64 vs bfloat16 (1/d^2 da xato), eager (cdist, kvadrat, teskari, matvec, bo'lish) vs kam oraliq massiv - GB/s.
6. (s6) Pool(1/2) to'r satr bo'laklari bo'yicha (20 bo'lak x 50 satr; stansiyalar fork orqali), chunksize 1/2/10; tezlanish, Pool xarajati, natija massivlarini birlashtirish (np.vstack) vaqti; natija tekshiruvi.
7. (s7) Amdahl: ketma-ket qism = xaritani PNG'ga render qilish (matplotlib imsave) + Python hisobot, parallel = prange IDW; S(2), p, S(8)/S(1024); Gustafson: to'r 200x200 -> 1 000x1 000 -> 2 000x2 000 da p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

13-variant. Sun'iy yo'ldosh tasvirlarida o'zgarishni aniqlash: oynali statistika integral tasvir bilan o'rta

Soha: Ekologiya

Tabiatni muhofaza qilish agentligi ikki sanadagi 2048x2048 sun'iy yo'ldosh tasvirini (1 kanal, aks ettirish) solishtirib o'rmon kesilishi va suv havzasi o'zgarishini topadi: har piksel atrofida 11x11 oynada o'rtacha va dispersiya hisoblanadi (ikkala tasvir uchun), so'ng normallashtirilgan farq va bo'sag'a qo'llanadi. Oynali statistika sodda yo'l bilan 121 o'qish/piksel (1 GFLOP), integral tasvir (kumulyativ yig'indi) bilan esa 4 o'qish/piksel - algoritmik tezlanish parallel tezlanishdan katta bo'lishi mumkin; kumulyativ yig'indi esa 'prefix sum' - parallellashtirish uchun qiyin klassik yadro. Tasvirlar sintetik: Perlin uslubidagi silliq maydon + o'zgargan bloklar + shovqin (seed 12).

Yadro	11x11 oynali o'rtacha/dispersiya (to'g'ridan-to'g'ri va integral tasvir) + normallashtirilgan farq + bo'sag'a
O'lchamlar	tasvir 512 / 1 024 / 2 048 (float32: 16 MB x 2), oyna 5 / 11 / 21
FLOP	to'g'ridan-to'g'ri: 2 x 121 x 4,2 M x 2 tasvir = 2 GFLOP (2048); integral: ~20 FLOP/piksel
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Blok	satr bo'laklari 32 / 128 / 512 (halo bilan)
Vaqt chegarasi	2048 juft tasvir <= 1 s
Ma'lumot	sintetik silliq maydon + o'zgargan bloklar, seed 12

QADAMLAR

1. (s1) Sof Python oynali o'rtacha (4 sikl) 256x256, oyna 5 vs NumPy: scipy.ndimage.uniform_filter va integral tasvir (np.cumsum ikki o'q bo'ylab + 4 burchak) 512/1024/2048, oyna 5/11/21; ms, M piksel/s, tezlanish; integral usulda vaqt oyna o'lchamiga bog'liq emasligini jadvalda ko'rsating; maks xato (float64 integral vs float32 - katastrofik qisqarish!).
2. (s2) @njit to'g'ridan-to'g'ri oynali o'rtacha/dispersiya va prange satrlar bo'yicha 1/2 ip (oyna 11, 1024/2048); JIT vaqti, GFLOP/s, tezlanish; o'zgargan piksellar soni (bo'sag'adan oshganlar) prange reduktsiya 1 vs 2 ip.
3. (s3) Lokallik va algoritm: (a) to'g'ridan-to'g'ri oyna, (b) ajraladigan (11 gorizontal + 11 vertikal), (c) sirpanuvchi yig'indi (har qadam 1 qo'shish 1 ayirish), (d) integral tasvir - 1 ipda GFLOP/s va ms; satr bo'laklari 32/128/512 (halo 5 satr) ta'siri; qaysi usul xotira, qaysi hisob bilan cheklangan.
4. (s4) Oynali o'rtachani 4 usulda: Numba 1 ip (sirpanuvchi), prange 2 ip, NumPy integral tasvir, torch F.avg_pool2d(11, stride 1, padding 5) yoki conv2d 1/2 ip; dispersiya E[x^2]-E[x]^2; maks xato < 1e-3; jadval.
5. (s5) torch avg_pool2d 1 vs 2 ip (1024/2048), float32 vs float64 vs bfloat16; integral tasvir torch.cumsum ikki o'q (prefix sum CPU'da 1 ip!) 1 vs 2 ip; normallashtirilgan farq eager (6 amal) vs bitta ifoda vs Numba fused - GB/s.

6. (s6) Pool(1/2) tasvir plitkalari bo'yicha (16 plitka 512x512 + 5 piksel halo, har vazifa o'zgargan piksel maskasini qaytaradi), chunksize 1/2/8; tezlanish, IPC (512x512 bool = 256 KB) ulushi; plitka chegaralarida natija ketma-ket bilan bir xilligini tekshiring.
7. (s7) Amdahl: ketma-ket qism = cumsum (torch/numpy 1 ip) va bog'langan komponentlar (scipy.ndimage.label, 1 ip), parallel = prange statistika; profil bilan p va S(2) dan p; S(8)/S(1024); Gustafson: 512 -> 2048 -> 4096 (1 tasvir 64 MB, ehtiyot) p; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

14-variant. Futbol o'yinchilari traektoriyasini Kalman filtri bilan silliqlash (ko'p obyektli kichik matritsalar) o'rta

Soha: Sport

Sport analitika kompaniyasi o'yin videosidan olingan 22 o'yinchi + to'p koordinatalarini (25 kadr/s, 90 daqiqa = 135 000 kadr) Kalman filtri bilan silliqlaydi va tezlik/tezlanishni baholaydi: holat 4 o'lchamli (x, y, vx, vy), har qadamda bir nechta 4x4 va 4x2 matritsa amallari. Bu 'ko'p mayda matritsa' (batched small matmul) yadrosi: BLAS 4x4 uchun samarasiz, Python sikli esa 3 M qadam uchun juda sekin; vaqt o'qi bo'ylab bog'liqlik bor (parallellashmaydi), obyektlar bo'ylab mustaqil (23 ta - ip sonidan ko'p, lekin uncha ko'p emas). Trayektoriyalar sintetik: silliq harakat + o'lchov shovqini (seed 13).

Yadro	Kalman bashorat + yangilash: F x, F P F^T + Q, K = P H^T (H P H^T + R)^-1 (4x4, 4x2, 2x2 teskari)
O'lchamlar	obyekt 23 / 230 (10 o'yin birga), qadam 13 500 / 135 000
FLOP	~600 FLOP/qadam x 23 x 135 000 = 1,9 GFLOP
Iplar/jarayonlar	Numba 1/2, torch bmm 1/2, Pool 1/2
Paket	obyektlar paketi 1 / 23 / 230 (torch bmm uchun)
Vaqt chegarasi	1 o'yin (23 x 135 000) <= 2 s
Ma'lumot	sintetik traektoriyalar + shovqin, seed 13

QADAMLAR

1. (s1) Sof Python Kalman (ro'yxatlar bilan 4x4 amallar qo'lda yozilgan) 1 obyekt x 13 500 qadam vs NumPy (har qadamda np kichik massivlar - 'NumPy mayda massiv' xarajati!) vs NumPy vektorlangan obyektlar bo'yicha (23 x 4 x 4 einsum/matmul); ms, M qadam/s, tezlanish; maks xato.
2. (s2) @njit Kalman (4x4 amallar skalyar yoki kichik massiv) 1 obyekt va prange obyektlar bo'yicha 1/2 ip (23 va 230 obyekt); JIT vaqti; M qadam/s, tezlanish; o'rtacha tezlik va maksimal tezlanish prange reduktsiya 1 vs 2 ip; 23 obyekt 2 ipga qanday taqsimlanadi (12+11) - yuk muvozanati.
3. (s3) Lokallik: holat va kovariatsiyani (obyekt, 4) / (obyekt, 4, 4) (AoS) vs (4, obyekt) / (4, 4, obyekt) (SoA - ichki sikl obyektlar bo'ylab vektorlashadi) joylashuvida o'lchang; qadamlarni 100/1 000/13 500 lik bo'laklarda ishlab (o'lchov massivi keshda) GFLOP/s; jadval.
4. (s4) Bir qadamni 230 obyekt uchun 4 usulda: Numba 1 ip, prange 2 ip, NumPy np.matmul batched (230x4x4 @ 230x4x4) + np.linalg.inv 2x2 batched, torch.bmm 1/2 ip (+ torch.linalg.inv); 135 000 qadam uchun jami; maks xato < 1e-6; qaysi usulda 'bir qadam xarajati' (Python chaqiruv) ustun.
5. (s5) torch bmm zanjiri 1 vs 2 ip (23/230/2 300 obyekt), float32 vs float64 (kovariatsiya barqarorligi!), eager (8 ta bmm/add alohida) vs oldindan hisoblangan doimiy F P F^T + Q bloklari bilan kamroq chaqiruv - M qadam/s; 'batch kattalashgani sari vaqt/qadam/obyekt' grafigi.

6. (s6) Pool(1/2) o'yinlar bo'yicha (10 o'yin x 23 obyekt, har vazifa o'z traektoriyasini seed bilan hosil qiladi va tezlik statistikasini qaytaradi), chunksize 1/2/5; tezlanish, Pool xarajati; obyektlar bo'yicha bo'lish (23 vazifa) bilan solishtiring.
7. (s7) Amdahl: vaqt o'qi ketma-ket, obyektlar parallel - S(2) dan p; obyekt soni 23 -> 230 -> 2 300 da S(2) va p qanday o'sadi (Gustafson: ko'proq o'yin birga); S(8)/S(1024) - GPU'da 23 obyekt bilan nima bo'ladi (band bo'lmagan yadrolar); grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

15-variant. Video quvuri: 1080p kadrlarni 720p ga bilinear kichraytirish va RGB -> YUV konvertatsiya oson

Soha: Media

Onlayn video platformasi yuklangan rolik kadrlarini (1920x1080 RGB) oldindan ko'rish uchun 1280x720 ga kichraytiradi va kodlashdan oldin YUV (BT.709) fazosiga o'tkazadi. Bir rolik 500 kadr; 30 kadr/s real vaqtdan tez (ya'ni <= 33 ms/kadr) ishlanishi kerak. Bilinear interpolyatsiya - har chiqish pikseli uchun 4 ta kirish pikseli (gather), RGB -> YUV esa har pikselga 3x3 matritsa (elementlararo GEMM) - ikkalasi ham xotira bilan cheklangan; siz birlashtirish (ikkisini bitta o'tishda) va iplar/jarayonlar ta'sirini o'lchaysiz. Kadrlar sintetik gradient + harakatlanuvchi shakllar (seed 14); xotirada 20 kadr (124 MB), qolgani jarayon ichida hosil qilinadi.

Yadro	bilinear 1920x1080 -> 1280x720 (3 kanal) + RGB -> YUV 3x3 matritsa + Y gistogrammasi (reduktsiya)
O'lchamlar	kadr 1080p (6,2 MB uint8), kadrlar 20 xotirada / 500 jami
Amallar	bilinear ~12 FLOP + YUV 18 FLOP = 30 FLOP x 0,92 M piksel = 28 MFLOP/kadr
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Paket	kadr paketi 1 / 5 / 20
Vaqt chegarasi	<= 33 ms/kadr
Ma'lumot	sintetik kadrlar, seed 14

QADAMLAR

1. (s1) Sof Python bilinear (ikki sikl, 1 kanal, 192x108 -> 128x72) vs NumPy (indeks massivlari + 4 ta gather + og'irliklar, fancy indexing) to'liq kadr; RGB -> YUV NumPy (reshape(-1,3) @ M.T) ; ms/kadr, M piksel/s, tezlanish; NumPy'dagi oraliq massivlar (float32 3 kanal x 4 gather) MB.
2. (s2) @njit bilinear + YUV bitta o'tishda (chiqish pikseli bo'yicha) va prange chiqish satrlari bo'yicha 1/2 ip; JIT vaqti; ms/kadr, GB/s (o'qish 4x3 bayt + yozish 3 bayt / piksel); Y gistogrammasi (256 bin) prange'da to'g'ri reduktsiya (har ip o'z gistogrammasi).
3. (s3) Lokallik: chiqish satri bo'yicha o'tish (kirishning 2 satri keshda) vs ustun bo'yicha; kadrlarni 1/5/20 lik paketlarda ishlab GB/s; uint8 -> float32 konvertatsiyani alohida o'tish vs ichkarida qilish; jadval.
4. (s4) Kichraytirish + YUV'ni 4 usulda: Numba 1 ip, prange 2 ip, NumPy gather + GEMM (3x3), torch F.interpolate(mode='bilinear', align_corners=False) + einsum 1/2 ip (paket 5 kadr); maks xato <= 1 (uint8) - align_corners va yarim-piksel kelishuvini moslashtiring; jadval.
5. (s5) torch interpolate + YUV 1 vs 2 ip (paket 1/5/20), float32 vs float16 vs uint8->float32 vaqti; YUV eager (3 ta kanal ifodasi alohida) vs einsum/matmul bitta chaqiruv vs Numba fused - GB/s; antialias=True farqi.
6. (s6) Pool(1/2) roliklar bo'yicha (10 rolik x 50 kadr, har jarayon o'z kadrlarini seed bilan hosil qiladi va Y gistogrammasini qaytaradi), chunksize 1/2/5; tezlanish, samaradorlik; kadrlarni pickle bilan yuborish (6 MB) vs seed yuborish IPC farqi - o'lchab ko'rsating.

7. (s7) Amdahl: ketma-ket qism = kadrni PNG'ga yozish (1 ip, matplotlib/PIL) yoki metadata Python, parallel = prange kichraytirish; S(2), p, S(8)/S(1024); Gustafson: 720p -> 1080p -> 4K (1 kadr 25 MB) chiqishda p; grafik; 33 ms/kadr talabini qaysi usul bajaradi.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha usullar bir xil natija beradi) — 30, O'lchovlar va jadval (metodika to'g'ri) — 25, Tahlil: tezlanish sabablari, Amdahl bahosi — 25, Hisobot sifati — 20

16-variant. Sayohat tavsiya tizimi: ALS matritsa faktorizatsiyasini foydalanuvchilar bo'yicha parallellashtirish qiyin

Soha: Turizm

Sayohat platformasi 50 000 foydalanuvchi x 5 000 mehmonxona/tur reytinglarini (2 M baho, siyrak) rank-32 ALS (Alternating Least Squares) bilan faktorlaydi. Har iteratsiyada har foydalanuvchi uchun o'zi baholagan k_u ta obyekt vektorlaridan Gram (32x32) yig'ilib, 32x32 chiziqli tizim yechiladi; so'ng obyektlar uchun xuddi shunday. Foydalanuvchilar mustaqil (prange), lekin har biri uchun ish hajmi har xil (k_u 5 dan 500 gacha) - yuk muvozanati masalasi; Gram yig'ish - mayda matmul, solve - LAPACK. 10 iteratsiya 20 s ichida tugashi kerak. Reytinglar sintetik: yashirin faktorlar + shovqin, k_u Zipf taqsimotida (seed 15).

Yadro	ALS: har foydalanuvchi uchun A = V_u^T V_u + lambda I (32x32), b = V_u^T r_u, solve; so'ng obyektlar uchun
O'lchamlar	foydalanuvchi 5 000 / 50 000, obyekt 5 000, rank 16 / 32, baholar 200 k / 2 M (CSR)
FLOP	Gram: 2 x 2 M x 32^2 = 4,1 GFLOP; solve 50 000 x (2/3) 32^3 = 1,1 GFLOP har iteratsiya
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2
Rejalashtirish	prange statik vs foydalanuvchilarni k_u bo'yicha saralab taqsimlash
Vaqt chegarasi	10 iteratsiya <= 20 s
Ma'lumot	yashirin faktorlar + Zipf k_u, seed 15

QADAMLAR

1. (s1) Sof Python ALS foydalanuvchi qadami (ro'yxatlar, Gram 3 sikl, Gauss yo'q qilish) 100 foydalanuvchi vs NumPy (V[idx].T @ V[idx], np.linalg.solve) 5 000/50 000; ms, GFLOP/s, tezlanish; RMSE har iteratsiyada kamayishini jadvalda ko'rsating (to'g'rilik).
2. (s2) @njit ALS foydalanuvchi qadami (CSR indptr/indices/data, Gram ikki sikl, np.linalg.solve njit ichida) va prange foydalanuvchilar bo'yicha 1/2 ip; JIT vaqti (LAPACK chaqiruvi bilan); GFLOP/s, tezlanish; RMSE prange reduktsiya 1 vs 2 ip; 2 ipda tezlanish 2 dan past bo'lsa k_u notekisligi sababini profil bilan ko'rsating.
3. (s3) Lokallik va muvozanat: foydalanuvchilarni k_u bo'yicha kamayish tartibida saralab prange'ga berish vs tasodifiy tartib; V matritsasini (obyekt, rank) C-tartibda vs F-tartibda; Gram'ni V[idx] nusxalab GEMM (BLAS) vs nusxasiz to'plash (gather) - rank 16/32 uchun jadval.
4. (s4) Bir foydalanuvchi guruhi (1 000 ta) uchun Gram + solve 4 usulda: Numba 1 ip, prange 2 ip, NumPy (padding bilan batched: k_max gacha nol to'ldirilgan (1000, k_max, 32) tensor va np.matmul + np.linalg.solve batched), torch bmm + torch.linalg.solve 1/2 ip; maks xato < 1e-5; padding qancha ortiqcha FLOP keltiradi - hisoblang.
5. (s5) torch batched solve 1 vs 2 ip (1 000/10 000 foydalanuvchi), float32 vs float64 (A shartliligi, lambda ta'siri); Gram eager (transpose, bmm, add lambda) vs torch.baddbmm bitta chaqiruv - GFLOP/s; bfloat16 Gram xatosi.

6. (s6) Pool(1/2) foydalanuvchi bloklari bo'yicha (10 blok x 5 000; CSR va V fork orqali meros, natija U bloki 5 000 x 32 float32 = 640 KB), chunksize 1/2/5; tezlanish; Numba prange ichidagi LAPACK iplari + 2 jarayon = ortiqcha obuna - OMP_NUM_THREADS/numba.set_num_threads(1) bilan solishtiring.
7. (s7) Amdahl: foydalanuvchi qadami (parallel) + obyekt qadami (parallel) + RMSE va Python boshqaruv (ketma-ket): profil ulushlari va S(2) dan p; k_u notekisligi tufayli 'yuk muvozanati yo'qotishi' ni Amdahl'ning ketma-ket qismi kabi ko'rsating; S(8)/S(1024); Gustafson: 5 000 -> 50 000 foydalanuvchi; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; Yuk muvozanati grafigi: har ipga tushgan k_u yig'indisi (statik vs saralangan taqsimot)

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

17-variant. Qurilish konstruksiyasining chekli elementlar modeli: siyrak SpMV va konjugat gradientlar qiyin

Soha: Qurilish

Loyiha instituti ko'p qavatli bino karkasining issiqlik/deformatsiya maydonini 100x100x100 = 1 M tugunli 3D to'rda modellashtiradi; natijaviy tizim siyrak (har qatorda 7 nol bo'lmagan element, CSR, 7 M nnz) va konjugat gradientlar (CG) usuli bilan yechiladi: har iteratsiya 1 SpMV + 2 skalyar ko'paytma + 3 AXPY. SpMV - xotira bilan cheklangan yadro (AI ~0,15 FLOP/bayt): 7 M nnz x (4 bayt qiymat + 4 bayt indeks) + vektorlar = ~70 MB har iteratsiya, hisob esa atigi 14 MFLOP. Shuning uchun 2 ip tezlanishi DRAM o'tkazuvchanligi bilan cheklanadi, format (float32/float64, int32 indeks) va qatorlarni qayta tartiblash (lokallik) esa real ta'sir beradi. 100 iteratsiya 3 s ichida bajarilishi kerak. Matritsa sintetik: 3D 7 nuqtali Laplas + diagonal ustunlik (seed 16).

Yadro	CSR SpMV y = A x + CG iteratsiyasi (dot, axpy); 1 M x 1 M, 7 nnz/qator
O'lchamlar	to'r 32^3 / 64^3 / 100^3 (nnz 0,23 M / 1,8 M / 7 M)
FLOP va bayt	SpMV 14 MFLOP, ~70 MB trafik (float32 + int32) -> AI ~0,2
Iplar/jarayonlar	Numba 1/2, torch.sparse 1/2, scipy.sparse (1 ip), Pool 1/2
Blok	qator bloklari 1 000 / 10 000 / 100 000; ELL/CSR format
Vaqt chegarasi	100 CG iteratsiyasi (100^3) <= 3 s
Ma'lumot	7 nuqtali Laplas + diagonal, seed 16

QADAMLAR

1. (s1) Sof Python CSR SpMV (indptr sikli) 32^3 vs scipy.sparse.csr_matrix @ x va NumPy 'stensil shaklida' (7 ta siljitilgan massiv qo'shish, matritsasiz) 32^3/64^3/100^3; ms, GFLOP/s va GB/s, tezlanish; CSR vs stensil xotira trafigi (indeks saqlanmaydi!) farqi.
2. (s2) @njit CSR SpMV va prange qatorlar bo'yicha 1/2 ip; JIT vaqti; GB/s samarali (nnz x 8 + 2 x n x 4 bayt / vaqt) - DRAM chegarasi bilan solishtiring (A2 2-qadam reduktsiya natijasi); CG'dagi dot va norma prange reduktsiya 1 vs 2 ip; 100 iteratsiya CG qoldiq egri chizig'i (to'g'rilik: ||r|| < 1e-6).
3. (s3) Lokallik: tugunlarni tabiiy (i,j,k) vs tasodifiy almashtirilgan tartibda raqamlash - SpMV vaqti (x ga murojaat lokalligi); float64 + int64 vs float32 + int32 formatlar; ELLPACK (n x 7 zich) vs CSR; qator bloklari 1 000/10 000/100 000 (prange chunk) ta'siri; jadval + GB/s.
4. (s4) SpMV'ni 4 usulda: Numba 1 ip, prange 2 ip, scipy.sparse (1 ip), torch.sparse_csr_tensor @ x 1/2 ip (yoki torch stensil: F.conv3d 3x3x3 yadro 7 nuqta - 'matritsasiz' variant); maks xato < 1e-5; qaysi usul GB/s bo'yicha DRAM chegarasiga eng yaqin.
5. (s5) torch CG (sparse mm + dot + axpy) 1 vs 2 ip (64^3/100^3), float32 vs float64 (CG yaqinlashuvi!); AXPY'lar eager (3 alohida) vs torch.add(alpha=) in-place vs Numba fused 'SpMV + dot' (bir o'tishda) - GB/s; iteratsiya vaqtida SpMV vs vektor amallari ulushi.
6. (s6) Pool(1/2) qator bloklari bo'yicha SpMV (A va x fork orqali meros, natija y bloki qaytadi) - tezlanish va nima uchun 100 iteratsiya CG'ni Pool bilan qilish befoyda (har iteratsiyada x ni

jarayonlarga yuborish 4 MB); muqobil: 8 ta mustaqil yuk holati (o'ng tomon) ni Pool'da bo'lish - tezlanish.

7. (s7) Amdahl: CG'da parallel (SpMV, prange) va ketma-ket (skalyar alpha/beta hisobi, Python sikl boshqaruvi, 100 x 2 bar'yer) ulushlari; S(2) dan p va profil p; S(8)/S(1024) - xotira chegarasi tufayli Amdahl bashorati nima uchun optimistik (GB/s chegara modelini qo'shing); Gustafson: 32^3 -> 100^3 -> 160^3 (float32, 29 M nnz, ehtiyot) ; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; Roofline-uslubidagi nuqta: SpMV AI va erishilgan GFLOP/s ni CPU xotira chegarasiga nisbatan (grafik)

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

18-variant. Kon seysmik monitoringi: 3D to'lqin tenglamasini 7 nuqtali stensil bilan modellashtirish qiyin

Soha: Ko'mir / kon sanoati

Ko'mir koni geofizika xizmati tog' jinslarida seysmik to'lqin tarqalishini 192x192x192 3D to'rda (7 nuqtali Laplas stensili, ikkinchi tartibli vaqt sxemasi) 50 vaqt qadamida modellashtiradi: har qadam 7 M katak x 10 FLOP, ikki bufer almashinadi. Portlash ishlaridan keyin natija 10 s ichida kerak. 3D stensil - GPU'dagi klassik 'shared memory tiling' yadrosi; CPU'da u xotira bilan cheklangan (AI ~0,6), lekin 3D bloklash (kesh plitkalari) va vaqt bo'yicha bloklash AI'ni oshiradi. Z o'qi bo'ylab prange yuk muvozanati va halo almashinuvini ko'rsatadi. Tezlik modeli sintetik: 3 qatlam + qazilgan bo'shliq, manba Ricker impulsi (seed 17).

Yadro	3D 7 nuqtali stensil: u_new = 2u - u_old + c^2 dt^2 (lap u) + manba; energiya reduktsiyasi
O'lchamlar	to'r 64^3 / 128^3 / 192^3 (float32: 28 MB x 3 bufer), qadam 50
FLOP	10 FLOP x 7,1 M x 50 = 3,5 GFLOP (192^3)
Iplar/jarayonlar	Numba 1/2, torch conv3d 1/2, Pool 1/2 (Z bo'yicha domen)
Blok	3D plitka 8 / 16 / 32 (y-z tekislikda), vaqt bloki 1 / 2
Vaqt chegarasi	192^3 x 50 qadam <= 10 s
Ma'lumot	3 qatlam tezlik modeli + bo'shliq, Ricker manba, seed 17

QADAMLAR

1. (s1) Sof Python 3D stensil 32^3 x 5 qadam (3 sikl) vs NumPy slicing (6 ta siljitilgan massiv) 64^3/128^3/192^3 x 50 qadam; ms, M katak/s, GFLOP/s, tezlanish; NumPy'da har qadam yaratiladigan oraliq massivlar (6 x 28 MB) va samarali GB/s; energiya saqlanishi tekshiruvi (to'g'rilik).
2. (s2) @njit stensil (i, j, k sikllari, k ichki - ketma-ket xotira) va prange eng tashqi (i) o'q bo'yicha 1/2 ip; JIT vaqti; GFLOP/s va GB/s (7 o'qish + 1 yozish + u_old); to'lqin energiyasi sum(u^2) prange reduktsiya 1 vs 2 ip; prange'ni ichki o'qqa ko'chirib nima bo'lishini o'lchang.
3. (s3) 3D bloklash: y-z plitkalari 8/16/32 (plitka x 3 tekislik x 4 bayt keshda), sikl tartibi (i-j-k vs k-j-i), vaqt bo'yicha bloklash 2 qadam (halo 2 katak); har varianti uchun GFLOP/s va erishilgan AI; 'plitka hajmi - L2 kesh' jadvali; 192^3 da bloklash foyda beradimi.
4. (s4) Stensilni 4 usulda: Numba 1 ip, prange 2 ip, NumPy slicing, torch F.conv3d (3x3x3 yadro, 7 nol bo'lmagan koeffitsiyent) 1/2 ip; 50 qadamdan keyin maydon maks farqi < 1e-3 (float32); torch conv3d 27 ko'paytma bajaradi (FLOP 4x ko'p) - lekin GFLOP/s qanday.
5. (s5) torch conv3d 1 vs 2 ip (64^3/128^3/192^3), float32 vs float64 vs bfloat16 (50 qadamda dispersiya xatosi); vaqt sxemasi eager (2u, -u_old, c^2 lap, + manba alohida) vs torch.addcmul / bitta ifoda vs Numba fused - GB/s.
6. (s6) Pool(1/2) Z (i) o'qi bo'yicha domen dekompozitsiyasi: 2 yarim to'r, har qadamda halo tekisligi (192x192x4 bayt = 147 KB) mp.Pipe orqali almashinadi; tezlanish va aloqa ulushi; muqobil: 8 ta mustaqil manba joylashuvini Pool'da bo'lish (p ~ 1) - solishtiring.

7. (s7) Amdahl: ketma-ket qism = manba qo'shish + chegara (PML soddalashtirilgan) Python'da har qadam + halo almashinuvi, parallel = prange stensil; S(2) dan p va profil p; S(8)/S(1024) va xotira chegarasi tuzatishi; Gustafson: 64^3 -> 192^3 -> 256^3 (3 x 64 MB) ; grafik; GPU (RTX 3060, 360 GB/s) uchun kutilgan M katak/s bashorati.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'; 3D plitka sxemasi (matn/rasm) va 'plitka hajmi - kesh' jadvali

Baholash (100 ball): Kod: parametrlangan yadrolar, bloklash/parallellashtirish to'g'ri — 25, O'lchovlar va Roofline-uslubidagi (AI vs GFLOP/s) tahlil — 20, Grafiklar va o'lcham/blok/iplar bo'yicha tadqiqot — 20, Muhandislik xulosasi: qaysi daraja qachon, GPU bashorati, Amdahl — 25, Hisobot sifati — 10

19-variant. Kredit skoring: logistik regressiyani gradient tushishi bilan o'qitish (matvec + fused sigmoid) o'rta

Soha: Bank

Bankning kredit bo'limi 500 000 mijoz x 64 belgi (daromad, qarz yuki, tarix...) ma'lumotida logistik regressiya modelini to'liq gradient tushishi bilan qayta o'qitadi (haftasiga, 200 epoxa). Har epoxa: z = X w (matvec), p = sigmoid(z), gradient X^T (p - y), log-loss (reduktsiya) - ya'ni 2 ta tor GEMV (AI past: 64 FLOP/qator per 256 bayt) + elementlararo amallar. Bu yadro xotira bilan cheklangan (X 128 MB har epoxa ikki marta o'qiladi), shuning uchun X'ni bloklab bir o'tishda ikkala GEMV'ni va sigmoid'ni birlashtirish (fused) asosiy tezlanish manbai. 200 epoxa 60 s ichida tugashi kerak. Ma'lumot sklearn make_classification (500 000 x 64, class_sep 1,0, seed 18); to'g'rilik - sklearn LogisticRegression koeffitsiyentlari bilan solishtirish.

Yadro	GEMV X w, sigmoid, GEMV X^T r, log-loss reduktsiyasi (epoxa)
O'lchamlar	mijoz 50 000 / 500 000, belgi 32 / 64 (float32: 128 MB), epoxa 20 / 200
FLOP	4 x 500 000 x 64 = 128 MFLOP/epoxa, trafik 256 MB/epoxa -> AI 0,5
Iplar/jarayonlar	Numba 1/2, torch 1/2, Pool 1/2 (bo'laklar bo'yicha gradient)
Blok	qator bloklari 1 000 / 10 000 / 100 000
Vaqt chegarasi	200 epoxa (500 000 x 64) <= 60 s
Ma'lumot	sklearn make_classification, seed 18

QADAMLAR

1. (s1) Sof Python epoxa (ikki sikl, math.exp) 2 000 x 64 vs NumPy (X @ w, 1/(1+np.exp(-z)), X.T @ r) 50 000/500 000 x 32/64; ms/epoxa, GFLOP/s va GB/s, tezlanish; 20 epoxadan keyin loss va sklearn bilan koeffitsiyent farqi; NumPy'da sigmoid 4 ta oraliq massiv (500 000 x 4 bayt) trafigi.
2. (s2) @njit epoxa (qator bo'yicha: z_i, p_i, r_i va darhol gradientga qo'shish - X bir marta o'qiladi!) va prange qatorlar bo'yicha 1/2 ip (gradient 64 ta akkumulyator - har ip o'z massivida, so'ng yig'iladi); JIT vaqti; ms/epoxa, GB/s; log-loss prange reduktsiya 1 vs 2 ip.
3. (s3) Lokallik: X C-tartib (qator = mijoz) vs F-tartib (ustun = belgi) da GEMV va GEMV^T tezligi; qator bloklari 1 000/10 000/100 000 (blok x 64 x 4 bayt) bilan 'blok ichida ikkala GEMV' - X bir o'tishda; jadval + GB/s; float32 vs float64 trafik farqi.
4. (s4) Epoxani 4 usulda: Numba 1 ip (fused), prange 2 ip, NumPy BLAS (2 GEMV + sigmoid), torch (X @ w, torch.sigmoid, X.T @ r) 1/2 ip; 20 epoxadan keyin w maks farqi < 1e-4; jadval; qaysi variantda X necha marta o'qiladi.
5. (s5) torch epoxa 1 vs 2 ip (50 000/500 000), float32 vs bfloat16 X (xotira 2x kam - tezlashadimi? xato?) vs float64; sigmoid + loss eager (6 amal) vs F.binary_cross_entropy_with_logits (fused) vs Numba fused - GB/s; torch.compile yo'q - izoh.
6. (s6) Pool(1/2) ma'lumot parallelligi: X bo'laklari bo'yicha qisman gradient (10 bo'lak x 50 000, X fork orqali meros, natija 64 float + loss), chunksize 1/2/5, 20 epoxa (har epoxada Pool qayta ishlatiladi - yaratmang!); tezlanish; bu DDP all-reduce'ning CPU muqobili ekanini tushuntiring.

7. (s7) Amdahl: ketma-ket qism = w yangilash + loss log + early-stopping tekshiruvi (Python, har epoxa), parallel = prange epoxa; S(2) dan p, S(8)/S(1024); xotira chegarasi: GB/s o'lchangan vs reduktsiya chegarasi; Gustafson: 50 000 -> 500 000 -> 2 M mijoz (bloklab generatsiya, 512 MB - ehtiyot) ; grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10

20-variant. Murojaatlar dublikatini topish: n-gramm bitset'lari ustida Jaccard o'xshashligi (AND + popcount) o'rta

Soha: Davlat xizmatlari

Davlat xizmatlari portaliga kuniga minglab murojaat keladi; takroriy va nusxa ko'chirilgan murojaatlarni topish uchun har matn 3-gramm to'plamiga, so'ng 2 048 bitli bitset'ga (32 ta uint64 so'z, xesh bilan) aylantiriladi va barcha juftliklar uchun Jaccard ~ popcount(A & B) / popcount(A | B) hisoblanadi. 8 000 murojaat uchun 32 M juft x 32 so'z = 1 G bit amal - sof Python to'plamlar bilan bu daqiqalar, Numba bitset bilan soniya; bu butun sonli (bit) yadro, FLOP emas - unumdorlikni 'juft/s' va 'GB/s' da o'lchaysiz. Matnlar sintetik: 50 ta shablon + tasodifiy so'z almashtirishlar (seed 19), shuning uchun haqiqiy dublikat juftlar ma'lum va to'g'rilik tekshiriladi.

Yadro	bitset AND/OR + popcount barcha juftliklar; Jaccard >= 0,8 juftlarni sanash (reduktsiya)
O'lchamlar	murojaat 1 000 / 8 000, bitset 1 024 / 2 048 bit (16 / 32 uint64 so'z)
Amallar	8 000^2/2 = 32 M juft x 32 so'z x 3 amal ~ 3 G butun son amal
Iplar/jarayonlar	Numba 1/2, torch 1/2 (bit amallari uint8/int64), Pool 1/2
Blok	qator bloklari 100 / 500 / 2 000 (blok x 8 000 x 1 bayt natija)
Vaqt chegarasi	8 000 murojaat <= 3 s
Ma'lumot	50 shablon + so'z almashtirish, seed 19

QADAMLAR

1. (s1) Sof Python: to'plamlar (set & set) bilan Jaccard 1 000 murojaat (0,5 M juft) vs Python int bitset (bin(a & b).count('1')) vs NumPy uint8 bitset (np.bitwise_and + np.unpackbits/popcount jadvali, bloklab) 1 000/8 000; ms, M juft/s, tezlanish; dublikat juftlar soni haqiqiy bilan mos (to'g'rilik).
2. (s2) @njit bitset Jaccard (popcount: 'bit hiylasi' SWAR formulasi yoki 16 bitli jadval) ikki sikl va prange qatorlar bo'yicha 1/2 ip (ichki sikl j > i - uchburchak, yuk muvozanati!); JIT vaqti; M juft/s, GB/s; dublikatlar soni prange reduktsiya 1 vs 2 ip; prange schedule muvozanatini i tartibini aralashtirib tekshiring.
3. (s3) Lokallik: bitset'larni (n, 32) uint64 C-tartibda vs so'z-birinchi (32, n) tartibda; qator bloklari 100/500/2 000 (blok x 256 bayt keshda, ichki sikl barcha j bo'ylab); 1 024 vs 2 048 bit; popcount jadval (64 KB, keshda) vs SWAR (hisob) - qaysi biri tez; jadval.
4. (s4) Jaccard'ni 4 usulda: Numba 1 ip, prange 2 ip, NumPy bloklab (bitwise_and + popcount jadval), torch (bitwise_and int64 + popcount jadval orqali yoki unpackbits muqobili: uint8 ko'rinishida (n, 256) va 256 li jadval indekslash) 1/2 ip; muqobil 'GEMM hiylasi': 0/1 float32 matritsa (n x 2 048) uchun A @ A.T = kesishma soni - BLAS bilan; natijalar mosligi (dublikat to'plami tengligi).
5. (s5) torch 1 vs 2 ip (1 000/8 000), int64 bitset vs uint8 vs 0/1 float32 GEMM vs bfloat16 GEMM (kesishma sonlari butun - xato bormi?); Jaccard eager (and, or, popcount, popcount, bo'lish) vs |A|+|B|-|A&B| hiylasi bilan bitta popcount - GB/s.

6. (s6) Pool(1/2) qator bloklari bo'yicha (16 blok x 500 qator; bitset'lar fork orqali meros; natija - dublikat juftlar ro'yxati), chunksize 1/2/8; uchburchak ish taqsimoti tufayli muvozanat - bloklarni 'oldingi + oxirgi' juftlab bering; tezlanish, samaradorlik.
7. (s7) Amdahl: ketma-ket qism = matnlarni n-grammga va bitset'ga aylantirish (Python, 8 000 matn) + dublikat guruhlarini birlashtirish (union-find, Python), parallel = prange Jaccard; profil ulushlari va S(2) dan p; S(8)/S(1024); Gustafson: 1 000 -> 8 000 -> 30 000 murojaat (ekstrapolyatsiya); grafik.

Topshiriladi: Kod (.py yoki .ipynb): har usul alohida funksiya, bir xil o'lchov metodikasi (isinish + takror + median); Natijalar jadvali: usul, o'lcham, iplar/jarayonlar, ms, GFLOP/s yoki M element/s, tezlanish, maks xato; Grafiklar (PNG): tezlanish vs o'lcham, blok/bo'lak o'lchami ta'siri, Amdahl egri chiziqlari + o'lchangan nuqta; Xulosa (>= 150 so'z): qaysi daraja (vektorlash, iplar, bloklash, jarayonlar) qancha berdi va 'GPU bo'lsa nima o'zgaradi'

Baholash (100 ball): To'g'ri ishlaydigan kod (barcha qadamlar, etalon tekshiruvi) — 25, O'lchovlar, jadval va GFLOP/s yoki GB/s hisobi — 25, Grafiklar (tezlanish, blok ta'siri, Amdahl) — 15, Tahlil: xotira/hisob chegarasi, iplar samaradorligi, p bahosi — 25, Hisobot sifati — 10